



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

РАБОТА ДОПУЩЕНА К ЗАЩИТЕ

Заведующий
кафедрой _____ А.В. Юдин

«_____» _____ июня 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по направлению подготовки бакалавров 09.04.02 Информационные системы и
технологии

На тему: Разработка информационной системы управления средней авторемонтной
мастерской

Обучающийся _____
подпись

Фамилия Имя Отчество студента _____
Фамилия, имя, отчество

Шифр 0000000000
Группа ЭФО-06-22

Руководитель
работы _____
подпись

Ученая степень, ученое звание,
должность

Фамилия, имя, отчество

Москва 2026 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

СОГЛАСОВАНО

УТВЕРЖДАЮ

Заведующий
кафедрой _____

Директор
института _____

подпись

подпись

Юдин Александр Викторович

Пушкин Павел Юрьевич

Фамилия Имя Отчество

Фамилия Имя Отчество

«_____» _____ 20____ г.

«_____» _____ 20____ г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы бакалавра

Обучающийся	<u>Сатюков Дмитрий Георгиевич</u>	
	<i>Фамилия Имя Отчество</i>	
Шифр	<u>22Т0858</u>	
Направление		
подготовки	<u>09.03.02</u>	<u>Информационные системы и технологии</u>
	<i>код направления</i>	<i>наименование направления</i>
Группа	<u>ЭФБО-06-22</u>	

1. Тема выпускной квалификационной работы

Разработка информационной системы управления средней авторемонтной мастерской.

2. Цель и задачи выпускной квалификационной работы

Цель работы: Разработка и реализация информационной системы для автоматизации процессов управления средней авторемонтной мастерской, обеспечивающей повышение эффективности учёта заказов, управления ресурсами и взаимодействия с клиентами.

Задачи работы:

1. Провести анализ предметной области управления авторемонтной мастерской и обзор существующих программных решений.
2. Провести анализ и моделирование бизнес-процессов авторемонтной мастерской.
3. Сформировать требования к информационной системе управления авторемонтной мастерской.

4. Спроектировать архитектуру информационной системы и базу данных с учётом специфики предметной области.
5. Разработать модуль учёта клиентов и транспортных средств с хранением истории обращений и ремонтов.
6. Реализовать модули заказ-нарядов и складского учёта запчастей, включая формирование заказов на ремонт, составить тест-кейсы.
7. Провести тестирование системы и дать рекомендации по развертыванию и сопровождению.

3. Этапы выпускной квалификационной работы

№ этапа	Содержание этапа выпускной квалификационной работы	Результат выполнения этапа ВКР	Срок выполнения
1	Обоснование актуальности работы и подготовка введения	Введение	06.03.2026
2	Анализ предметной области авторемонтной мастерской, обзор существующих программных решений для автосервисов, моделирование бизнес процессов мастерской, формирование требований к системе	Глава 1	20.03.2026
3	Проектирование архитектуры информационной системы, интерфейса пользователя и базы данных	Глава 2	03.04.2026
4	Реализация всех модулей программного решения, составление тест-кейсов, тестирование, составить рекомендации по развертыванию и сопровождению, оценка результатов	Глава 3	17.04.2026
5	Формирование заключения и окончательное оформление выпускной квалификационной работы	Заключение, итоговый вариант ВКР	22.05.2026
6	Подготовка доклада и презентации к защите	Доклад, презентация	29.05.2026

4. Перечень разрабатываемых документов и графических материалов:

- пояснительная записка выпускной квалификационной работы;
- презентация;
- приложения.

5. Руководитель выпускной квалификационной работы

Функциональные обязанности	Должность в Университете	Фамилия Имя Отчество	Подпись
Руководитель ВКР	Доцент	Громов Евгений Владимирович	

Задание выдал
Руководитель ВКР:

Задание принял к исполнению
Обучающийся: _____

_____ подпись

_____ подпись

«_____» _____ 20____ г. «_____» _____ 20____ г.

Аннотация

Данная выпускная квалификационная работа посвящена разработке информационной системы управления средней авторемонтной мастерской. Целью данной работы является автоматизация основных бизнес-процессов предприятия, а это включает учет заказов, управления ресурсами и конечно же взаимодействие с клиентами.

В ходе написания этой работы проведён анализ предметной области, а именно проанализированы особенности функционирования авторемонтных мастерских и выполнен обзор существующих программных решений. Также стоит отметить, что выполнено моделирование ключевых бизнес-процессов, таких как приём заказов, диагностика, ремонт и выдача автомобилей.

На основе проведённого анализа сформированы основные требования к системе. Была разработана архитектура информационной системы и спроектирована база данных. Также были реализованы модули: учёт клиентов и транспортных средств, управление заказ-нарядами и складской учет запасных частей.

Значимость данной работы и в конечном итоге разработки такой системы, заключается в повышении эффективности управления мастерской за счёт автоматизации процессов.

Оглавление

Введение	7
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ РАЗРАБОТКИ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ АВТОРЕМОНТНОЙ МАСТЕРСКОЙ.....	9
1.1. Особенности деятельности авторемонтных мастерских	9
1.2. Проблемы управления авторемонтными мастерскими без информационных систем	11
1.3. Обзор существующих информационных систем для управления автосервисами	13
1.4. Сравнительный анализ существующих решений	15
1.5. Обоснование необходимости разработки новой информационной системы	16
1.6. Анализ и моделирование бизнес-процессов авторемонтной мастерской.....	17
1.7. Постановка задачи для разработки информационной системы для автомастерской	19
Выводы	21
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ АВТОРЕМОНТНОЙ МАСТЕРСКОЙ.....	22
2.1. Краткая характеристика исследуемого объекта	22
2.2. Описание основных бизнес-процессов	22
2.2. Процесс обработки заказ-наряда.....	25
2.3. Процесс управления складскими остатками.....	27
2.4. Постановка задач и формирование требований к разрабатываемому решению.....	29
2.5. Архитектура приложения	31
2.6. Проектирование интерфейса	32
2.7. Проектирование модели данных.....	34
2.8. Выбор и обоснование технологического стека	36
2.9. Проектирование подсистемы защиты персональных данных	38
Выводы	41
ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ АВТОСЕРВИСОМ	42
3.1. Реализация серверной части.....	42
3.1.1. Middleware аутентификации и авторизации	42
3.2. Реализация базы данных	44
3.3. Реализация бизнес-логики заказ нарядов.....	45
3.4. Реализация складского учёта.....	47
3.6. Сервис E-mail уведомлений.....	48
3.7. Реализация подсистемы защиты персональных данных	48
3.8. Реализация клиентской части.....	50

3.9. Тестирование системы	51
Список использованных литературных источников.....	53
Приложения	55
Приложение А - Сравнительная таблица существующих решений.....	55
Приложение Б - Диаграммы бизнес-процессов.....	56
Приложение В – Бриф проекта	58
Приложение Г – Use Case Диаграммы	60
Приложение Д – Интерфейс. Проектирование.....	64
Приложение Е – Таблицы сравнений технологий.....	67
Приложение Ё – Модели данных.....	69
Приложение Ж – Реализация и демонстрация кода.....	69

Введение

В современном мире, где цифровизация проникает даже в самые традиционные сферы бизнеса, именно по этой причине автосервисы уже не могут эффективно работать, основывая все свои процессы на ручном труде, бумажных журналах и разрозненных Excel-таблицах. В данной сфере можно заметить, как в небольших мастерских механики тратят часы на поиск запчастей в таблицах, приёмщики переписывают заказ-наряды от руки, а владелец пытается вручную посчитать, сколько на самом деле заработано и какая у мастерской выручка. Все эти факты приводят к ошибкам, потерянными клиентам, простоям и упущенной прибыли. Именно поэтому была выбрана данная тема выпускной квалификационной работы.

Актуальность данной темы обусловлена растущей потребностью автосервисных предприятий в средствах автоматизации. К примеру, автомобильный парк России устойчиво увеличивается: по данным аналитического агентства «Автостат», к концу 2024 года он превысил 45 миллионов легковых автомобилей [1], а это ведёт к пропорциональному росту спроса на услуги технического обслуживания и ремонта. Значительную долю этого рынка занимают небольшие мастерские [3]. Однако возрастающий поток заказов порождает сложности в управлении: ведение записей клиентов, контроль хода работ, формирование документов и взаимодействие с заказчиками требуют значительных временных затрат при ручной обработке. Одновременно растут и ожидания клиентов-потребителей всё чаще рассчитывают на оперативные уведомления о статусах ремонта, электронные счета и прозрачную отчетность. Исходя из всего этого в условиях цифровизации экономики автосервисы, не использующие информационные системы, рискуют утратить конкурентоспособность [14].

Объектом исследования в данной работе является деятельность средней авторемонтной мастерской.

Предметом исследования выступает информационная система автоматизации управления основными бизнес-процессами средней автомастерской.

Целью данной выпускной квалификационной работы является разработка и реализация информационной системы для автоматизации процессов управления средней авторемонтной мастерской, обеспечивающей повышение эффективности учёта заказов, управления ресурсами и взаимодействия с клиентами. Для достижения поставленной цели будет необходимо решить и реализовать следующие задачи: провести анализ предметной области управления авторемонтной мастерской и обзор существующих программных решений; провести анализ и моделирование бизнес-процессов авторемонтной мастерской; сформировать требования к информационной системе управления авторемонтной мастерской; спроектировать архитектуру информационной системы и базу данных с учётом специфики предметной области; разработать модуль учёта клиентов и транспортных средств с хранением истории обращений и ремонтов; реализовать модули заказ-нарядов и складского учёта запчастей, включая формирование заказов на ремонт, составить тест-кейсы; провести тестирование системы и дать рекомендации по развертыванию и сопровождению.

Данная работа сочетает в себе как теоретический анализ предметной области, так и практическую разработку программного продукта, который может быть реально внедрён в небольшом автосервисе.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ РАЗРАБОТКИ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ АВТОРЕМОНТНОЙ МАСТЕРСКОЙ

1.1. Особенности деятельности авторемонтных мастерских

Авторемонтные мастерские – это один из самых «живых» и востребованных сегментов бизнеса. Во время сбора информации и материалов можно быстро понять: здесь всё построено на людях, скорости и доверии. Клиент приезжает не просто починить машину – он хочет уехать в хорошем настроении, без потраченных лишних сил и непонятных цифр в счёте. Именно поэтому особенности деятельности таких мастерских сильно отличаются от крупных дилерских центров.

По данным агентства «Автостат», на конец года парк легковых автомобилей в России достиг примерно 47,5 миллионов единиц [1]. Основываться вся статистика будет на нашем регионе, потому что он ближе всего к нам. Это огромный и постоянно растущий рынок. А ёмкость автосервиса в 2025 году составила 1 триллион 204,1 миллиарда рублей – это почти на 20 процентов больше, чем годом ранее. В стране работает больше 130 тысяч точек обслуживания и ремонта легковых автомобилей [3]. При этом официальные дилеры занимают всего около 4-8 процентов рынка услуг, а независимые СТО – около 32-37 процентов, ну и самая большая группа (63 %) – это как раз небольшие и средние мастерские с ограниченным набором услуг и небольшой производственной площадью.

Средняя авторемонтная мастерская с персоналом примерно до 20 человек (именно такой формат был выбран для исследования) обычно выглядит так. Владелец часто совмещает роль директора и главного человека, через которого проходят большинство процессов бизнеса. У него в подчинении 1-2 приёмщика заказов, 8-10 механиков, кладовщик и, как

правило, бухгалтер на аутсорсе. Всё производство делится на зоны: приёмка, диагностика, ремонтные посты, склад запчастей и небольшая клиентская зона.

Если говорить об основных бизнес-процессах, то они в основном очень простые, но зависят от человеческого фактора:

- Приём автомобиля и оформление заказа-наряда;
- Диагностика (часто может проходить вручную или со сканером);
- Планирование работ и распределение по механикам;
- Поиск, закупка и учёт запчастей;
- Собственно, сам ремонт и контроль качества;
- Расчёт с клиентом и выдача машины.

Самая большая особенность таких мастерских – это до сих пор преобладающий ручной или полу ручной учёт. Заказ-наряды часто пишут от руки или в Excel, остатки на складе держат в тетрадке или в голове у кладовщика. Если добавить к этому сезонные пики, постоянную нехватку времени на поиск деталей и жёсткую конкуренцию: с одной стороны – крупные сети или дилеры, с другой – самозанятые мастера, которые работают дешевле и быстрее.

Всё это делает работу мастерской очень гибкой, но очень уязвимой. Одна ошибка в учёте запчастей – и клиент уйдёт недовольный качеством услуг и скорее всего больше не обратится в данный сервис. Ошибки в планировании могут вызывать проблемы с выручкой. Именно поэтому нужна простая, понятная и недорогая информационная система, которая уберёт всю бумажную рутину и даст владельцу реальный контроль над бизнесом. Вот такие особенности деятельности авторемонтных мастерских будут рассмотрены подробно дальше в данной главе.

1.2. Проблемы управления авторемонтными мастерскими без информационных систем

Отсутствие специализированных информационных систем в управлении средней авторемонтной мастерской приводит к систематическому накоплению операционных, финансовых и конкурентных рисков. В процессе анализа предметной области и изучения отраслевых источников можно выявить ряд ключевых проблем, характерных для независимых несетевых предприятий, использующих преимущественно ручные методы учёта и планирования.

Одной из наиболее значимых проблем является высокая вероятность ошибок в учёте заказов, запасных частей и финансовых операций. Ручное ведение заказов, а также хранение данных о складских остатках в «уме» или в неструктурированных записях приводит к пересортам, недостаткам и перерасходу материалов. По предварительным оценкам можно сказать, что внедрение автоматизированного складского учёта позволяет значительно сократить количество ошибок и время на выполнение операций. Без таких инструментов возникают простои производственных постов из-за отсутствия необходимых деталей и несоответствия фактических и учётных остатков.

Также стоит обратить внимание, что непрозрачность финансовых потоков и утечки прибыли становятся следствием отсутствия централизованного контроля. Владелец предприятия не получает оперативной информации о реальной марже по каждому заказу, объёме дебиторской задолженности, скрытых расходах или незафиксированных случаев дополнительных работах. Такие факторы указывают, что это приводит к потере потенциальной выручки из-за человеческого фактора – забытых платежей, неучтенных услуг или некорректного распределения затрат. Отсутствие аналитики по ключевым показателям затрудняет принятие обоснованных управленческих решений.

Третья группа проблем связана с неэффективным планированием загрузки и распределением задач. При использовании бумажных графиков или общих календарей часто возникают простои механиков, перегрузка в пиковые периоды и несогласованность работ. Согласно опросам независимых СТО, коммерческие трудности и недостаток клиентов остаются ведущими вызовами, а неоптимальная организация процессов усугубляет ситуацию, снижая общую производительность.

Четвертый аспект – это снижение качества клиентского обслуживания и утрата лояльности. Современные потребители ожидают прозрачности: предварительной сметы, фото/видеоотчётов о диагностике, уведомлений о статусе работ и истории обслуживания в единой системе. Ручные методы не позволяют оперативно предоставлять такую информацию, что приводит к расхождениям между озвученной и фактической стоимостью, задержкам и негативным отзывам. В результате наблюдается отток клиентов к более организованным конкурентам.

В условиях 2025-2026 годов указанные проблемы приобретают особую остроту на фоне рыночных изменений. Ёмкость рынка автосервисных услуг в России по итогам в 2025 года достигла 1 триллион 204,1 миллиарда рублей, однако количество точек сократилось на 6 процентов после пика в 2024 году [2]. Это свидетельствует о естественном исчезновении с рынка слабых бизнесов, не сумевших адаптироваться к растущим требованиям клиентов, повышению цен на услуги и усилению конкуренции. Игнорирование цифровизации процессов лишает независимые мастерские конкурентных преимуществ в скорости, прозрачности и контроле затрат.

Данный анализ показывает, что отсутствие информационной системы не только снижает операционную эффективность, но и создаёт прямую угрозу устойчивости бизнеса в условиях высокой конкуренции и изменяющихся рыночных условий. Разработка специализированного решения для средних

мастерских представляется необходимым направлением для минимизации указанных рисков.

1.3. Обзор существующих информационных систем для управления автосервисами

В рамках подготовки данной работы был проведён анализ современных информационных систем и программных продуктов, предназначенных для автоматизации процессов в автосервисах и станциях технического обслуживания. Особое внимание уделялось решениям, адаптированным для независимых средних мастерских, где приоритетными факторами выступают простота внедрения, наличие облачных версий и соответствие типичным операционным задачам.

По состоянию на 2025–2026 годы рынок специализированного ПО для автосервисов характеризуется доминированием отечественных разработок, преимущественно на базе платформы 1С, а также облачных SaaS-решений. Зарубежные системы имеют ограниченное распространение ввиду языковых и регуляторных барьеров, а также отсутствия локализованной технической поддержки.

Можно выделить следующие наиболее востребованные продукты:

АвтоДилер Онлайн – одна из лидирующих облачных систем для независимых СТО, присутствующая на рынке уже большое количество лет [6]. Функционал включает оформление заказ-нарядов, встроенные нормы времени работ (в том числе на кузовные и покрасочные работы), складской учёт с автоматической оценкой по более чем 150 поставщикам, CRM-модуль, онлайн-запись, интеграцию с мессенджерами, расчёт заработной платы механикам, поддержку маркировки «Честный знак» и выгрузку данных в 1С [7]. Решение ориентировано на малый и средний бизнес благодаря минимальным требованиям к инфраструктуре и возможности начала работы в течение нескольких минут. Имеется мобильное приложение для механиков и

приёмщиков, позволяющее оформлять акты осмотра и диагностические листы непосредственно в ремонтной зоне [7].

1С:Автосервис – отраслевое решение на платформе «1С:Предприятие 8», созданное на основе конфигурации «1С:Управление нашей фирмой» [8]. Обеспечивает полную интеграцию с «1С:Бухгалтерией», детальный контроль запасных частей с использованием специализированных справочников (модели автомобилей, цеха, виды ремонта), нормирование работ с интеграцией с каталогами Autodata, AutoSoft и SilverDat [9]. Система реализует поэтапное выполнение ремонта с формированием печатных форм (акт осмотра, рабочий лист, заказ-наряд, квитанция), планирование загрузки цехов, анализ рентабельности, а также встроенную CRM с коммуникациями через email и IP-телефонию [9]. Основные ограничения для средних мастерских – высокая сложность внедрения, необходимость привлечения сертифицированного специалиста 1С и значительная совокупная стоимость владения (лицензии, доработки, сопровождение).

STOCRM – специализированная облачная CRM-система для автосервисов и магазинов автозапчастей [10]. Система построена на принципах организации производства канбан и реализует полноценную воронку продаж, управление задачами в формате канбан-доски, заказ-наряды со встроенными нормами времени, оценку запчастей у поставщиков, складской учёт с базой более 9 млн позиций, календарь загрузки постов, расчёт заработной платы, онлайн-запись и интеграции с 1С Бухгалтерией, IP-телефонией, мессенджерами [11]. Имеется мобильное приложение.

Splus – облачная программа для автоматизации автосервиса, работающая на рынке с 2013 года [12]. Включает заказ-наряды, справочник нормативов времени от автопроизводителей, интеграцию с более чем 50 поставщиками запчастей, складской учёт, расчёт заработной платы, планировщик загрузки постов и клиентское приложение «Сервисная книжка»

[13]. Система ориентирована на малый и средний автосервис и отличается простотой освоения.

1.4. Сравнительный анализ существующих решений

На основе проведенного обзора были выделены шесть ключевых систем, наиболее релевантных задачам средней автосервисной мастерской: АвтоДилер Онлайн, 1С:Автосервис, Splus и STOCRM. Для объективного сопоставления были определены критерии, отражающие основные функциональные и эксплуатационные требования автосервиса. Результаты представлены в сравнительной таблице в приложении 1.

На основе сформированной таблицы можно сделать следующие выводы. Ряд функций присутствует в большинстве рассмотренных систем и является фактическим стандартом для данного класса программного обеспечения в сфере автомастерских. К ним относятся: оформление заказ-нарядов, ведение клиентской базы, онлайн-запись, уведомление клиентов, удаленный доступ и мобильное приложение. Указанные функции являются базовыми и должны быть реализованы в разрабатываемой системе.

Вместе с тем ряд важных для автосервиса возможностей представлен лишь в части решений. К примеру, в АвтоДилер онлайн отсутствует отслеживание истории ремонта того или иного автомобиля, что будет добавлено в разрабатываемое решение. Из особенностей, которые можно выделить и добавить в задачи разработки – это взаимодействие с клиентом, отслеживание статуса ремонта и демонстрация при помощи фото состояния автомобиля. Такие функции есть STOCRM.

Отдельного внимания заслуживает система STOCRM, которая, в отличие от остальных рассмотренных решений, реализует полноценную воронку продаж и управление задачами в формате канбан-доски. Данный подход основан на системе организации производства Toyota и обеспечивает прозрачность рабочих процессов и контроль сроков выполнения работ. Для

данного проекта можно взять в учёт именно реализацию канбан-доски, которая отлично подойдет для данной сферы автомастерских.

1.5. Обоснование необходимости разработки новой информационной системы

Проведённый в подразделе 1.4 сравнительный анализ существующих информационных систем (таблица 1) показал, что все перечисленные решения обладают преимуществами, способны закрыть проблему автоматизации для больших автомастерских. Но в данной работе рассматривается всё же средняя автомастерская, для которой функционал определённых решений является громоздким. Для средней автомастерской подходила бы в основном только SPLUS, но в ней отсутствует определенное количество функций, по типу истории ремонта автомобиля, как и в АвтоДилер онлайн, а также нормального планирования работы, как это реализовано в STOCRM.

Разработка новой информационной системы направлена на устранение указанных проблем. Набор функциональных требований к проектируемой системе сформирован строго на основе результатов анализа, проведённого в подразделе 1.4, и объединяет в себе несколько групп возможностей:

- Базовые функции: оформление заказ-нарядов, ведение клиентской базы, онлайн-запись, уведомления клиентов через Email и удаленный облачный доступ;
- Уникальные и оптимизирующие функции (встречающиеся в аналогах лишь единично): управление задачами в формате канбан-доски, формирование истории ремонта автомобилей, формирование отчётов во всех подходящих форматах, которые помогут администратору анализировать продажи.

1.6. Анализ и моделирование бизнес-процессов авторемонтной мастерской

Типовой основной бизнес-процесс в такой мастерской – это полный цикл обслуживания клиента от момента первого контакта до выдачи отремонтированного автомобиля. Он включает несколько ключевых этапов:

- Предварительная запись и первичный контакт – клиент звонит, пишет в мессенджер или приходит без записи. Приёмщик фиксирует марку/модель авто, год выпуска, пробег, описанную проблему и желаемое время визита. Часто это происходит в телефоне или простом Google-календаре;
- Приёмка автомобиля – клиент приезжает, приёмщик проводит внешний осмотр, фиксирует видимые повреждения (часто фото на телефон), пробег, комплектацию, жалобы владельца. Составляется заказ-наряд (от руки или в Excel), обсуждается предварительная стоимость и сроки. Автомобиль ставится на мойку или сразу на пост диагностики;
- Диагностика – механик проводит визуальный осмотр, компьютерную диагностику (OBD-сканером), проверку систем (двигатель, подвеска, тормоза и т.д.). Формируется дефектная ведомость с перечнем необходимых работ и запчастей. Всё это согласовывается с клиентом (по телефону или лично);
- Планирование и закупка запчастей – если детали отсутствуют на складе, приёмщик/кладовщик делает заказ у поставщиков. Механики распределяются по постам (в общем календаре). Работы начинаются после одобрения сметы клиентом;
- Выполнение ремонта/ТО – механики проводят работы по нормам времени. В процессе могут возникать дополнительные работы, которые снова согласовываются;

- Контроль качества и сдача работ – после ремонта автомобиль проверяется (тест-драйв, повторная диагностика), клиент приглашается на приёмку. Подписывается акт выполненных работ, выдаётся счёт;
- Выдача автомобиля и расчёт – клиент оплачивает (наличными, картой), получает ключи, документы. Часто отправляется напоминание о следующем ТО.

В реальности эти этапы часто пересекаются, дублируются или пропускаются из-за нехватки времени. Например, диагностика может совмещаться с ремонтом, а закупка запчастей – затягиваться на дни, что приводит к простоям постов и недовольству клиентов.

Для моделирования были выбраны комбинации нотаций, которые чаще всего используется при разработке информационных систем: IDEF0 для функционального описания (верхний уровень) и BPMN для детального потока работ с ролями и решениями.

В IDEF0 на контекстном уровне (A-0) процесс выглядит как «Управление авторемонтной мастерской», с входами (автомобиль клиента, жалобы, запчасти), выходами (отремонтированный автомобиль, счёт, довольный клиент), механизмами (механики, оборудование, ПО) и управлением (правила, нормы, политика владельца).

На уровне A0 выделяются 4-5 основных блоков:

- A1 – Приёмка и диагностика;
- A2 – Планирование и закупки;
- A3 – Выполнение работ;
- A4 – Контроль и выдача.

В BPMN я описал основной процесс как последовательность задач с пулами ролей (Клиент, Приёмщик, Механик, Кладовщик, Владелец). Примеры элементов:

- Начало события – «Клиент обратился»;

- Задачи – «Оформить заказ-наряд», «Провести диагностику», «Согласовать смету»;
- Шлюзы – «Запчасть в наличии?» (да/нет – ветвление на закупку);
- Окончание события – «Автомобиль выдан».

Диаграммы данных бизнес-процессов представлены в приложении 2 №2.1-2.2.

1.7. Постановка задачи для разработки информационной системы для автомастерской

Анализ работы автомастерской показал несколько устойчивых проблем: запчасти учитываются вручную, из-за чего возникают пересортица и дефицит в самый неподходящий момент; заказы ведутся разрозненно, и руководителю сложно понять, что вообще происходит в мастерской; клиенты не получают никакой информации о ходе ремонта и вынуждены звонить сами. Также сравнительный анализ существующих решений привёл к выводам, что в разрабатываемой системе должны быть обязательно базовые функции, которые описаны в параграфе выше и несколько уникальных решений, которые реализованы не во всех рассмотренных продуктах.

Работать с системой будут пять категорий пользователей с разными правами и задачами: администратор (полный доступ ко всем разделам системы, управление пользователями и настройками), менеджер (работа с заказами, клиентами, аналитикой и отчётностью), мастер (ведение своих заказов, фиксация выполненных работ и загрузка фотоматериалов), кладовщик (управление складом, приём поставок, контроль остатков запчастей), клиент (отслеживание своих заказов, просмотр истории обслуживания и получение уведомлений).

Исходя из выявленных проблем и целей разработки, сформулированы первичные требования к системе.

Функциональные требования:

- система должна вести полный цикл заказ-наряда – от момента обращения клиента до выдачи автомобиля – с фиксацией каждого изменения;
- система должна автоматически списывать запчасти при добавлении их в заказ и предупреждать о том, что остаток опускается ниже допустимого минимума;
- клиент должен получать актуальную информацию о статусе своего заказа через уведомления внутри системы и на электронную почту;
- руководство должно иметь доступ к отчётам по заказам, мастерам, запчастям и клиентам;
- каждый пользователь должен видеть только те данные и функции, которые соответствуют его роли.

Нефункциональные требования:

- безопасность: пароли хранятся в хешированном виде, аутентификация построена на JWT-токенах, персональные данные защищены;
- доступность: система работает в браузере и не требует установки какого-либо дополнительного программного обеспечения.

В брифе, который находится в приложении 3, указано описание, цель, ожидаемые результаты.

Выводы

В результате первой главы были детально изучены особенности авторемонтной мастерской. Такие предприятия составляют самую многочисленную группу на рынке автосервиса. При этом основная нагрузка ложится на небольшой коллектив, где каждый сотрудник выполняет несколько ролей, а процессы все ещё во многом зависят от человеческого фактора.

Обзор существующих решений продемонстрировал, что готовые системы имеют системные ограничения для целевой группы: иногда высокая стоимость и сложность внедрения, недостаточная специализация под автосервис среднего размера. Ни одно из них не даёт оптимального сочетания простоты и полноты функционала именно для таких автомастерских, которые рассмотрены в данной работе.

По итогу в первой главе были сформулированы назначение системы, круг пользователей, её структура, а также подготовлена необходимая документация, а именно бриф проекта.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ АВТОРЕМОНТНОЙ МАСТЕРСКОЙ

2.1. Краткая характеристика исследуемого объекта

Объектом исследования является автосервис среднего масштаба, предоставляющий услуги по техническому обслуживанию и ремонту автомобилей. Предприятие обслуживает поток клиентов, обращающихся с различными запросами.

Организационная структура предприятия включает следующие категории персонала: администратор, осуществляющий полное управление системой (создание учётных записей сотрудников, управление справочниками услуг и запчастей, контроль бизнес-процессов); менеджер, работающий с клиентской базой, формирующий отчётность и контролирующий ход выполнения заказов; мастер – непосредственный исполнитель работ, создающий и ведущий заказ-наряды, назначающий услуги и запчасти, изменяющий статусы выполнения; кладовщик, управляющий складским учётом (приёмка запчастей от поставщиков, контроль остатков, формирование заказов на поставку); клиент – конечный потребитель услуг, имеющий возможность отслеживать статус своих заказов и получать уведомления через веб-интерфейс и по электронной почте.

2.2. Описание основных бизнес-процессов

В первой главе, а именно в параграфе 1.6, был рассмотрен полный цикл обслуживания клиента от момента первого контакта до выдачи отремонтированного автомобиля. Это основная функция автомастерской. Но стоит учитывать, что у каждой роли в системе тоже есть свои варианты использования и требования.

Для систематизации функциональных требований к системе была составлена карта пользовательских историй (User Story Map), отражающая

ключевые действия каждой роли. Истории сгруппированы по функциональным блокам.

Первый блок – это «Аутентификация и профиль»:

- Как клиент, я хочу зарегистрироваться в системе, чтобы в любой момент видеть, на каком этапе находится ремонт моего автомобиля;
- Как пользователь, я хочу входить в систему по email и паролю, чтобы сразу попадать в свой рабочий раздел;
- Как пользователь, я хочу редактировать своё имя, телефон, email и менять пароль, чтобы мои данные всегда были актуальны;
- Как клиент, я хочу самостоятельно включать и отключать email-уведомления, чтобы получать только нужные мне оповещения.

Второй блок – это «Управление заказ-нарядами»:

- Как мастер или администратор, я хочу создавать заказ-наряд, указывая клиента, его автомобиль и описание проблемы, чтобы зафиксировать обращение;
- Как мастер или администратор, я хочу назначать конкретного исполнителя на заказ, чтобы было понятно, кто отвечает за работу;
- Как мастер или администратор, я хочу добавлять услуги из каталога в заказ, чтобы итоговая сумма считалась автоматически без ручных вычислений;
- Как мастер, администратор или кладовщик, я хочу добавлять запчасти в заказ так, чтобы они сразу списывались со склада;
- Как мастер или администратор, я хочу переводить заказ между статусами с обязательным комментарием, чтобы была понятна причина каждого изменения;
- Как клиент, я хочу видеть все свои заказы и полную историю смены статусов по каждому из них, чтобы всегда быть в курсе происходящего;

- Как администратор, я хочу удалять заказ в статусе «Принят» или «Отменён», при этом использованные запчасти должны автоматически вернуться на склад.

Третий блок – это «Транспортные средства»:

- Как мастер или администратор, я хочу добавлять автомобиль клиента с указанием марки, модели, года выпуска, VIN и госномера, чтобы вести точный учёт техники;
- Как мастер, менеджер или администратор, я хочу открывать карточку автомобиля и видеть всю историю его обслуживания: какие работы выполнялись и какие запчасти использовались.

Четвёртый блок – это «Каталог услуг»:

- Как администратор, я хочу вести каталог услуг с наименованием, категорией, базовой ценой и примерной длительностью, чтобы мастера могли быстро выбирать нужные позиции;
- Как администратор, я хочу деактивировать услугу, чтобы она пропала из каталога, но при этом осталась в истории прошлых заказов.

Пятый блок – это «Складской учёт»:

- Как кладовщик или администратор, я хочу вести каталог запчастей с артикулами, ценами и текущими остатками, чтобы всегда знать, что есть на складе;
- Как кладовщик или администратор, я хочу видеть список запчастей, остаток которых упал ниже минимального порога, чтобы вовремя делать заказ поставщику;
- Как кладовщик или администратор, я хочу оформлять поступление товара от поставщика так, чтобы остатки на складе обновлялись автоматически;

- Как кладовщик или администратор, я хочу вести справочник поставщиков с их контактами и ИНН, чтобы нужная информация всегда была под рукой.

Шестой блок – это «Аналитика и отчётность»:

- Как менеджер или администратор, я хочу видеть на дашборде сводку по заказам и загруженность каждого мастера, чтобы оперативно оценивать ситуацию;
- Как менеджер или администратор, я хочу формировать отчёты по заказам с фильтрацией по датам, статусам и выручке, чтобы анализировать работу за нужный период;
- Как менеджер или администратор, я хочу видеть отчёт по выполненным работам в разрезе каждого мастера, чтобы оценивать их производительность;
- Как менеджер или администратор, я хочу формировать отчёт по расходу запчастей, чтобы контролировать затраты и планировать закупки;
- Как менеджер или администратор, я хочу видеть отчёт по клиентам с суммой их трат и количеством автомобилей, чтобы понимать, кто является ключевым клиентом.

Представленные пользовательские истории охватывают все роли, определённые в параграфе, и формируют основу для последующего проектирования диаграмм использования. На основании пользовательских историй выделены следующие актёры и сценарии использования: клиент, мастер, кладовщик, менеджер, администратор – наследует возможности всех ролей. Диаграммы находятся в Приложение Г.

2.2. Процесс обработки заказ-наряда

Центральным бизнес-процессом системы является жизненный цикл заказ-наряда. Заказ проходит через шесть статусов, каждый переход

фиксируется в журнале аудита и сопровождается уведомлением клиента. Схема переходов между статусами представлена на Рисунке 2.2.1, также в Приложении А, Рисунок 2.3.

На начальном этапе заказ-наряд получает статус «Принят». Мастер или администратор создаёт запись с указанием клиента, автомобиля, описания проблемы и, при необходимости, ответственного мастера. Клиенту автоматически направляется уведомление о создании заказа.

После проведения диагностики мастер определяет перечень необходимых работ и запчастей, формирует предварительную стоимость, и заказ переводится в статус «Ожидает согласования». На данном этапе клиент подтверждает или корректирует предложенный объём работ. Если клиент согласовал работы, но на складе отсутствуют необходимые запчасти, заказ переходит в статус «Ожидает запчастей». Кладовщик формирует заказ поставщику, и после поступления запчастей на склад заказ может быть переведён на следующий этап. Статус «В работе» означает, что все запчасти в наличии и мастер приступил к выполнению ремонтных работ. По завершении всех работ заказ переводится в статус «Завершён», при этом автоматически фиксируется дата завершения. Клиент получает уведомление о готовности автомобиля к выдаче.

На любом этапе жизненного цикла заказ может быть переведён в статус «Отменён» мастером или администратором.

Диаграмма состояний заказ-наряда



Рисунок 2.2.1 – Диаграмма состояний заказ-наряда.

2.3. Процесс управления складскими остатками

Вторым по значимости бизнес-процессом является управление складскими запасами. Система должна реализовать два основных механизма

изменения остатков: списание при использовании запчастей в заказе и приход при оформлении поставки от поставщика.

Списание со склада происходит при добавлении запчасти в заказ-наряд. Пользователь (мастер, кладовщик или администратор) выбирает запчасть из каталога и указывает необходимое количество. Система выполняет проверку текущего остатка: если доступное количество на складе меньше запрашиваемого, операция отклоняется с уведомлением о недостаточном количестве запчастей. В случае успешной проверки создаётся запись о привязке запчасти к заказу с фиксацией цены на момент использования, остаток на складе уменьшается на указанное количество с применением атомарной операции, исключающей ошибки при одновременном доступе нескольких пользователей, а итоговая стоимость заказа автоматически пересчитывается.

Приход на склад осуществляется при оформлении поставки. Кладовщик или администратор выбирает поставщика из справочника и формирует список позиций с указанием запчасти, количества и закупочной цены. После сохранения поставки остаток каждой указанной запчасти автоматически увеличивается на соответствующее количество.

Для предотвращения дефицита запчастей в системе должен быть реализован механизм контроля минимальных остатков. Каждая позиция каталога имеет параметр минимального порога запаса. Система предоставляет возможность отображения запчастей, текущий остаток которых ниже или равен установленному минимуму. Это позволяет кладовщику и менеджеру оперативно выявлять позиции, требующие пополнения, и своевременно формировать заказы поставщикам.

Кроме того, система должна обеспечивать корректный возврат запчастей на склад при удалении заказ-наряда. Данная операция допускается только для заказов в статусах «Принят» и «Отменён». При удалении система

обрабатывает все привязанные к заказу запчасти и возвращает указанные количества на склад, что гарантирует целостность складского учёта.

2.4. Постановка задач и формирование требований к разрабатываемому решению

Первичные требования, отражающие цели и назначение системы в целом, сформулированы в параграфе 1.7. Здесь они раскрываются подробнее – в разрезе конкретных функциональных модулей.

В практике разработки программного обеспечения принято разделять требования на два типа. Функциональные описывают, что именно система должна делать. Нефункциональные задают ограничения и качественные характеристики: насколько система безопасна, надёжна, удобна в использовании. Для данной системы ключевые нефункциональные требования – это безопасность обработки персональных данных, которая будет рассмотрена позже, и целостность данных и простота интерфейса.

Функциональные требования разбиты на десять модулей. Краткая характеристика каждого приведена ниже.

Аутентификация и авторизация – регистрация клиентов, вход по email и паролю, выдача JWT-токена сроком на 24 часа, разграничение доступа по пяти ролям.

Управление заказ-нарядами – полный жизненный цикл заказа с шестью статусами, фиксация каждого перехода, добавление услуг и запчастей, автоматический пересчёт итоговой стоимости.

Управление транспортными средствами – регистрация автомобилей с привязкой к владельцу, просмотр полной истории обслуживания, запрет удаления при наличии связанных заказов.

Каталог услуг – ведение каталога с категориями и ценами, деактивация услуг без удаления из истории заказов.

Складской учёт – учёт запчастей с текущими остатками, контроль минимального порога, автоматическое списание при добавлении в заказ.

Управление поставщиками – справочник поставщиков с контактами и ИНН, оформление поставок с автоматическим обновлением остатков на складе;

Уведомления – системные и email-уведомления при изменении статуса заказа, возможность отмечать уведомления как прочитанные.

Аналитика и отчётность – дашборд с загруженностью мастеров, отчёты по заказам, работам, запчастям и клиентам, экспорт в PDF.

Глобальный поиск - единая строка поиска по всем основным сущностям системы: заказам, клиентам, автомобилям, запчастям и услугам.

Фотогалерея заказов - загрузка фотографий с категоризацией по типу (до ремонта, после, общие), просмотр клиентом через личный кабинет.

Нефункциональные требования также разбиваются на несколько модулей.

Требования к безопасности:

- Обеспечение обязательной аутентификации для доступа ко всем функциональным возможностям системы, за исключением операций регистрации и входа;
- Защита учётных данных пользователей за счёт хранения паролей в зашифрованном виде и невозможности их восстановления;
- Контроль подлинности каждого запроса между клиентской и серверной частями;
- Автоматическое завершение пользовательской сессии при обнаружении несанкционированного доступа.

Требования к удобству использования (UX):

- Поддержка адаптивного интерфейса для различных устройств и разрешений экрана;
- Обеспечение визуальной обратной связи при загрузке и обработке данных;

- Единообразное представление информации и статусов во всех частях системы;
- Организация навигации с учётом ролей пользователей для упрощения доступа к функциональности.

2.5. Архитектура приложения

Система будет построена по клиент-серверной архитектуре с разделением на три слоя: клиентский слой, серверный слой и слой данных. Схема взаимодействия между слоями представлена на Рисунке 2.5.1. Данный подход обеспечивает модульность, масштабируемость и упрощает сопровождение системы за счёт разграничения ответственности компонентов.

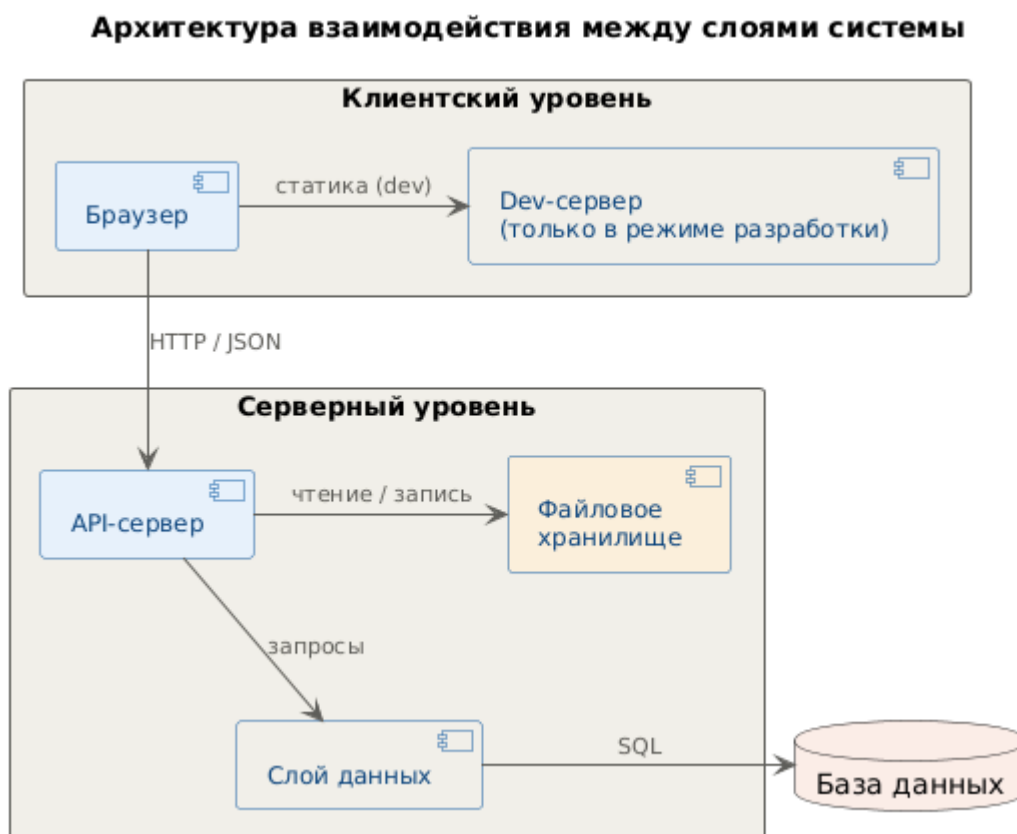


Рисунок 2.5.1 – Архитектура взаимодействия между слоями системы.

Клиентский слой проектируется в виде одностраничного веб-приложения, функционирующего в браузере пользователя. Он будет отвечать за отображение пользовательского интерфейса, обработку пользовательских действий, реализацию навигации, а также управление состоянием

приложения, включая аутентификацию пользователя и параметры интерфейса. Взаимодействие с серверным слоем будет осуществляться посредством HTTP-запросов с передачей данных в формате JSON. В рамках проектирования предусматривается двухуровневый механизм контроля доступа: на уровне маршрутизации будет выполняться проверка аутентификации и прав пользователя, а пользовательский интерфейс будет адаптироваться в зависимости от доступных ролей.

Серверный слой проектируется в виде приложения, реализующего программный интерфейс взаимодействия (API) по архитектурному стилю REST. Он будет обеспечивать обработку клиентских запросов, выполнение бизнес-логики, а также взаимодействие со слоем данных. Для обеспечения безопасности предусматривается механизм аутентификации на основе токенов и авторизация с использованием ролевой модели доступа. Дополнительно серверный слой будет включать подсистему отправки уведомлений по электронной почте и модуль обработки загружаемых файлов с их сохранением в файловой системе.

Слой данных проектируется на основе реляционной системы управления базами данных. Взаимодействие серверного слоя с базой данных будет осуществляться через специализированный программный слой абстракции доступа к данным. Для повышения эффективности использования ресурсов предусматривается применение паттерна Singleton, обеспечивающего использование единого экземпляра компонента доступа к данным в рамках серверного процесса.

2.6. Проектирование интерфейса

На этапе UX-проектирования, благодаря пользовательским историям и пользовательским сценариям, были определены основные категории пользователей системы: администратор, менеджер, мастер, кладовщик и клиент, а также их типовые действия в рамках работы с заказ-нарядами, клиентской базой, автомобилями, складом, поставками, отчетностью и

уведомлениями. Это позволило заложить в основу интерфейса ролевую логику доступа и выстроить навигацию не по формальному набору страниц, а по реальным бизнес-процессам автосервиса.

На основе всего этого анализа сформирован прототип с примерной структурой экранов, так называемых wireframes:

1. Первый экран – это всегда экран авторизации. Он предназначен для входа пользователя в систему по адресу электронной почты и паролю. После успешной аутентификации пользователь перенаправляется в раздел, соответствующий его роли. Для администратора и менеджера основной точкой входа является панель управления, для мастера и клиента - раздел заказ-нарядов.
2. Второй экран – это главный кран и представляет собой основной рабочий макет с боковой навигационной панелью и центральной областью отображения данных. Состав разделов в меню изменяется в зависимости от роли пользователя. В навигации предусмотрены переходы к заказ-нарядам, клиентам, автомобилям, услугам, запчастям, поставщикам, поставкам, отчетам, уведомлениям и профилю.
3. Главный экран используется руководящими ролями для быстрого получения сводной информации о состоянии автосервиса. На данном экране отображаются ключевые показатели, активные заказ-наряды, загрузка мастеров и другие оперативные данные. Из панели управления пользователь может перейти к детальному просмотру заказов и аналитических разделов.
4. Экран заказ-нарядов. Содержит перечень всех доступных пользователю заказов с основной информацией о клиенте, автомобиле, статусе и стоимости работ. Для удобства работы предусмотрен переход к созданию нового заказа, просмотру карточки заказа и альтернативному представлению в формате канбан-доски.

5. Канбан-доска. Предназначена для наглядного представления заказ-нарядов по стадиям выполнения. Заказы распределяются по колонкам в соответствии с их текущим статусом, что позволяет быстрее оценивать загрузку и текущее состояние работ. Данный экран особенно удобен для менеджера и мастера при оперативном контроле исполнения заказов.
6. Экран складского учёта. Он включает в себя разделы запчастей, поставщиков и поставок. На экране запчастей отображаются позиции склада, остатки и сведения о минимальном количестве.
7. Экран отчётности предназначен для того, чтобы формировать аналитическую информацию по заказам, работам и запчастям, а также по клиентам за выбранный период.
8. Отдельный экран с профилем пользователя. Для клиента предусмотрен отдельный вход, где он может просматривать статусы заказов и туда ему приходят уведомления. Также он там может изменить свои личные данные, либо удалить аккаунт, что предусмотрено нефункциональными требованиями.

Все схемы экранов находятся в Приложении

2.7. Проектирование модели данных

Модель данных проектируемой системы предназначена для хранения и обработки информации, связанной с пользователями, транспортными средствами, заказами, услугами, складскими запасами, поставками, а также уведомлениями и медиафайлами. При проектировании учитываются требования к целостности данных, их согласованности и возможности масштабирования при увеличении объёма информации.

В рамках модели данных выделяется группа сущностей, описывающих пользователей системы. Предусматривается хранение общих данных пользователей, а также разделение на роли, определяющие права доступа и функциональные возможности. Для отдельных категорий пользователей

проектируется возможность хранения дополнительных характеристик. Такой подход позволяет централизовать базовые данные и при этом учитывать специфику различных типов пользователей.

Группа, связанная с транспортными средствами, предназначена для хранения информации об объектах обслуживания. Каждый объект будет связан с конкретным пользователем системы. При проектировании учитывается необходимость уникальной идентификации транспортных средств и предотвращения дублирования записей. Объем данных данной группы относительно невелик, однако данные должны сохраняться длительное время в связи с необходимостью ведения истории обслуживания.

Центральное место в модели данных занимает группа, описывающая заказы. В ней предусматривается хранение информации о выполненных и текущих работах, включая связь с пользователями, транспортными средствами и ответственными сотрудниками, а также состояние выполнения. Дополнительно проектируется механизм фиксации изменений состояния заказа во времени для обеспечения аудита действий. Также предусматривается хранение информации о составе заказа, включая используемые услуги и материалы. При проектировании закладывается принцип сохранения исторических данных, в том числе значений стоимости на момент выполнения операций. Данная группа характеризуется значительным объемом данных и высокой интенсивностью изменений.

Группа справочных данных предназначена для хранения перечня услуг, доступных в системе. Для таких данных предусматривается возможность изменения их доступности без физического удаления, что позволяет сохранять целостность исторической информации. Объем данных данной группы невелик, изменения происходят относительно редко.

Группа, связанная со складом, включает информацию о материальных ресурсах, используемых в процессе выполнения заказов. При проектировании учитывается необходимость отслеживания текущего состояния запасов и

фиксации всех операций их изменения. Данные данной группы имеют средний объём и требуют обеспечения точности и согласованности.

Группа, связанная с поставщиками и поставками, предназначена для хранения информации о контрагентах и операциях поступления материальных ресурсов. Предусматривается связь между поставками и их составом, что позволяет отслеживать структуру каждой поставки. Объём данных данной группы является умеренным и зависит от интенсивности закупочной деятельности.

Группа уведомлений и медиа предназначена для хранения информации о взаимодействии системы с пользователями, а также сопутствующих данных, таких как файлы. При проектировании учитывается, что объём текстовых уведомлений невелик, тогда как объём медиафайлов может существенно возрастать. В связи с этим предусматривается разделение хранения самих файлов и их данных, связанных с описанием.

2.8. Выбор и обоснование технологического стека

Выбор конкретных технологий осуществляется на основе сформулированных функциональных требований (параграф 2.4) и архитектурных решений (параграф 2.5). Система строится по клиент-серверной архитектуре, что предполагает выбор технологий для четырёх ключевых компонентов: серверная платформа, система управления базами данных, ORM-библиотека и фреймворк клиентского приложения.

Для реализации серверной части были рассмотрены три платформы: Node.js с фреймворком Express, Python с фреймворком Django и Java с фреймворком Spring Boot. Сравнение проведено по критериям, наиболее значимым для разрабатываемой системы, и представлено в таблице 5.1 (Приложение 5).

В качестве серверной платформы выбран Node.js с фреймворком Express. Ключевым фактором является использование единого языка

TypeScript на стороне клиента и сервера, что снижает когнитивную нагрузку при разработке полного стека одним разработчиком. Встроенная асинхронная модель Node.js эффективна для задач, связанных с операциями ввода-вывода, характерных для разрабатываемой системы: обращение к базе данных, отправка email-уведомлений, обработка загрузки файлов. Express является минималистичным и гибким фреймворком, позволяющим точно контролировать архитектуру приложения без избыточных абстракций.

Для хранения данных были рассмотрены три СУБД: PostgreSQL, MySQL и MongoDB. Сравнение представлено в таблице 5.2 (Приложение 5).

Выбрана СУБД PostgreSQL. Предметная область автосервиса характеризуется чётко определёнными сущностями со множеством связей: клиент, автомобиль, заказ, услуги, запчасти, поставки. Реляционная модель PostgreSQL обеспечивает целостность данных на уровне СУБД через внешние ключи и ограничения уникальности (VIN-номер автомобиля, артикул запчасти, электронная почта пользователя). Кроме того, PostgreSQL поддерживает десятичный тип данных для точных финансовых расчётов без ошибок округления, что критически важно для корректного расчёта стоимости заказов и формирования отчётности. Документо-ориентированная СУБД MongoDB не подходит для данной задачи ввиду отсутствия поддержки внешних ключей и ограниченной поддержки транзакций, необходимых при одновременном изменении нескольких связанных сущностей (например, списание запчасти и пересчёт стоимости заказа).

Для взаимодействия серверного приложения с базой данных были рассмотрены три ORM-библиотеки для Node.js: Prisma, TypeORM и Sequelize. Сравнение представлено в таблице 5.3 (Приложение 5).

Выбрана библиотека Prisma. Она генерирует полностью типизированный клиент доступа к данным на основе декларативной схемы, что исключает несоответствие типов между кодом приложения и структурой базы данных на этапе компиляции. Механизм миграций автоматически

отслеживает изменения схемы и генерирует соответствующие SQL-скрипты, что упрощает развёртывание и сопровождение системы.

Для реализации пользовательского интерфейса были рассмотрены три фреймворка: React, Vue и Angular. Сравнение представлено в таблице 5.4 (Приложение 5).

Выбран React. Он обеспечивает компонентный подход к построению интерфейса с максимальной гибкостью выбора сопутствующих библиотек для маршрутизации, управления состоянием и построения UI-компонентов. Обширная экосистема и сообщество гарантируют наличие готовых решений для типичных задач. В сочетании с выбранной серверной платформой Node.js использование React позволяет вести всю разработку на едином языке TypeScript. Также создание большого количества интерактивных элементов по типу канбан-доски, легче реализуется при помощи React.

2.9. Проектирование подсистемы защиты персональных данных

Разрабатываемая информационная система обрабатывает персональные данные нескольких категорий субъектов. В процессе проектирования модели данных были выделены следующие категории обрабатываемых сведений:

- ФИО, адрес электронной почты, номер телефона, хеш пароля;
- VIN-номер, государственный регистрационный знак;
- Контактное лицо, номер телефона, адрес электронной почты, почтовый адрес, ИНН (Поставщики).

Поскольку перечисленные сведения относятся к персональным данным в соответствии с Федеральным законом от 27.07.2006 № 152-ФЗ «О персональных данных» [18], система должна обеспечивать их защиту. На этапе проектирования был сформирован перечень мер, которые необходимо реализовать на уровне приложения.

Пароли пользователей не должны храниться в открытом виде – это одно из базовых требований к безопасности любой системы. Для хеширования

выбран алгоритм bcrypt: он генерирует уникальную соль для каждого пароля и обеспечивает высокую вычислительную сложность подбора исходного значения. Благодаря этому даже при утечке базы данных восстановить пароли пользователей практически невозможно.

Фильтрация конфиденциальных полей в ответах API. Каждый контроллер, возвращающий сведения о пользователях, должен явно указывать, какие поля допустимо передавать клиенту. Хеш пароля не должен попадать ни в один ответ API – вне зависимости от контекста запроса.

Для аутентификации используется механизм JWT со сроком жизни токена 24 часа. В полезную нагрузку включаются только идентификатор пользователя и его роль – размещать в токене персональные данные (ФИО, email, телефон) не следует.

Доступ к персональным данным предоставляется по принципу минимально необходимых привилегий в рамках пятиуровневой модели ролей. Проверка прав осуществляется на двух уровнях: на сервере – через middleware-компоненты авторизации, на клиенте – через компонент защиты маршрутов.

На эндпоинты аутентификации устанавливается ограничение частоты запросов с одного IP-адреса. Мера направлена против двух типичных угроз: автоматизированного перебора паролей (brute-force) и массового создания учётных записей.

Серверное приложение должно выставлять защитные HTTP-заголовки: Content-Security-Policy – для защиты от XSS-атак; X-Frame-Options – для предотвращения clickjacking; X-Content-Type-Options – для блокировки MIME-подмены; Strict-Transport-Security – для принудительного использования HTTPS.

Ну и самое главное, система должна реализовывать право на удаление данных. Для реализации этого права выбрана стратегия анонимизации: персональные данные заменяются обезличенными значениями, тогда как

связанные записи – история заказов, финансовые данные – сохраняются для нужд бухгалтерского и управленческого учёта.

Выводы

Во второй главе выполнено проектирование веб-приложения информационной системы управления автосервисом.

На основе анализа деятельности средней автосервисной мастерской определены пять ролей пользователей системы (администратор, менеджер, мастер, кладовщик, клиент) и описаны выполняемые каждой ролью функции. Помимо основного бизнес-процесса обслуживания автомобиля, выявлены сопутствующие процессы: закупка запасных частей, выставление счетов и приём оплаты, расчёт заработной платы и формирование отчётности.

Для систематизации требований составлена карта пользовательских историй, распределённых функциональным блокам. Детально описаны два ключевых бизнес-процесса: жизненный цикл заказ-наряда, проходящего через шесть статусов с аудитом каждого перехода и автоматическим уведомлением клиента, а также процесс управления складскими остатками с механизмами списания, прихода и контроля минимальных порогов.

Также были сформулированы функциональные требования. На основе требований и с учётом архитектурных решений произведён обоснованный выбор технологического стека: серверная платформа Node.js с фреймворком Express, СУБД PostgreSQL, ORM-библиотека Prisma и библиотека React для клиентского приложения. Выбор каждой технологии обоснован сравнительным анализом альтернатив по ряду критериев.

ГЛАВА 3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ УПРАВЛЕНИЯ АВТОСЕРВИСОМ

3.1. Реализация серверной части

Точкой входа серверного приложения служит файл `server/src/index.ts`, в котором последовательно выполняется начальная конфигурация всех компонентов. На первом этапе производится загрузка переменных окружения из файла `«.env»`, что позволяет разделить конфигурационные параметры и исходный код приложения. Далее создаётся экземпляр веб-фреймворка с подключением необходимых промежуточных обработчиков: модуля для поддержки кросс доменных запросов и встроенного парсера JSON-тел входящих запросов.

Для обеспечения доступа к загруженным пользователями изображениям настраивается раздача статических файлов из каталога `uploads/` – каждый файл становится доступен по прямой URL без дополнительной обработки на стороне сервера. Маршрутизация запросов организована через 14 подключаемых модулей, зарегистрированных с базовыми путями `/api/auth`, `/api/orders`, `/api/parts` и другими. Дополнительно реализован служебный эндпоинт `/api/health`, предназначенный для проверки работоспособности сервера в рамках мониторинга.

3.1.1. Middleware аутентификации и авторизации

Разграничение доступа к ресурсам системы реализовано посредством двух промежуточных обработчиков, определённых в файле `server/src/middleware/auth.ts`.

Первый из них, `authenticate`, отвечает за идентификацию пользователя. Обработчик извлекает JWT-токен из заголовка запроса `Authorization: Bearer <token>`, верифицирует его подпись с использованием секретного ключа `JWT_SECRET` и декодирует полезную нагрузку, содержащую идентификатор пользователя и его роль. Полученные данные помещаются в объект `req.user`

для дальнейшего использования в цепочке обработки запроса. В случае отсутствия токена или его недействительности клиенту возвращается ответ с кодом 401.

Второй компонент, `authorize`, представляет собой функцию, принимающую перечень допустимых ролей в качестве аргументов. Формируемый ею обработчик проверяет соответствие роли аутентифицированного пользователя заданному списку и при несоответствии возвращает ответ с кодом 403.

Защита маршрутов организована на двух уровнях. На уровне маршрутного модуля обработчик `authenticate` применяется ко всем входящим запросам через `router.use(authenticate)`. На уровне отдельных эндпоинтов дополнительно используется `authorize` для ограничения доступа по роли:

Листинг 3.1 – Пример применения.

```
router.use(authenticate);  
router.post('/', authorize(Role.ADMIN, Role.MASTER), createOrder);  
router.patch('/:id/status', authorize(Role.ADMIN, Role.MASTER),  
updateOrderStatus);
```

Архитектура серверной части построена на принципе разделения ответственности между маршрутизацией и бизнес-логикой. Маршрутные модули, расположенные в каталоге `routes/`, определяют HTTP-метод, URL-путь и цепочку промежуточных обработчиков для каждого эндпоинта, после чего делегируют выполнение запроса соответствующей функции контроллера. Контроллеры, размещённые в каталоге `controllers/`, инкапсулируют прикладную логику: валидацию входных данных, взаимодействие с базой данных через ORM и формирование ответа. Каждая функция контроллера является асинхронной и имеет сигнатуру `(req: Request, res: Response) => Promise<void>`.

Всего в системе реализовано 14 контроллеров, соответствующих 14 маршрутным модулям.

3.2. Реализация базы данных

Реализация базы данных системы выполнена с использованием ORM-инструмента Prisma, который позволяет описывать структуру данных в декларативном виде и автоматически формировать соответствующую схему в реляционной системе управления базами данных. Описание схемы осуществляется в конфигурационном файле `schema.prisma`, который является центральной точкой определения моделей, связей и ограничений.

Модель данных представлена 15 сущностями, каждая из которых сопоставляется с отдельной таблицей в базе данных (Приложение Е, Рисунок 6.1). Для настройки соответствия между именами моделей в программном коде и именами таблиц в базе данных применяется директива «`@@map()`». Это позволяет использовать стиль именования `camelCase` на уровне приложения и `snake_case` на уровне базы данных, что соответствует распространённым практикам разработки.

Связи между сущностями реализованы с использованием механизма «`@relation`», обеспечивающего явное описание внешних ключей и зависимостей между таблицами. В рамках схемы определено 19 связей, отражающих основные отношения предметной области.

Для предотвращения дублирования данных в базе предусмотрены ограничения уникальности (`@unique`). Они применяются к ключевым атрибутам, таким как адрес электронной почты пользователя, идентификаторы, связанные с учетными записями, уникальные характеристики транспортных средств (VIN), а также артикулы материальных ресурсов.

Управление изменениями структуры базы данных осуществляется с использованием встроенного механизма миграций Prisma. При внесении изменений в файл схемы формируется SQL-миграция, которая применяется к базе данных и фиксируется в системе с версиями. Это позволяет отслеживать

историю изменений структуры и воспроизводить её на различных этапах разработки.

3.3. Реализация бизнес-логики заказ нарядов

Контроллер заказ-нарядов (`orderController.ts`) содержит наиболее сложную бизнес-логику системы и охватывает полный жизненный цикл заказа – от создания до удаления.

При создании нового заказ-наряда контроллер извлекает из тела запроса идентификаторы клиента, транспортного средства и мастера, а также описание работ. Заказу немедленно присваивается начальный статус “ACCEPTED”, и в рамках того же обращения к базе данных формируется первая запись в таблице истории статусов с комментарием «Заказ создан» и указанием автора изменения. Параллельно создаётся уведомление для клиента с информацией об автомобиле, а если у клиента включены email-уведомления, асинхронно – без блокировки основного потока выполнения – отправляется соответствующее письмо.

Листинг 3.3.1 – код уведомления клиенту

```
// Уведомление клиенту
await prisma.notification.create({
  data: {
    clientId,
    orderId: order.id,
    title: 'Новый заказ',
    message: `Создан заказ-наряд #${order.id} для вашего автомобиля
${order.vehicle.brand} ${order.vehicle.model}`,
  },
});

// Email-уведомление
const clientUser = await prisma.user.findFirst({
  where: { client: { id: clientId } },
  select: { email: true, emailNotifications: true },
});
if (clientUser?.emailNotifications) {
  sendNewOrderEmail(
```

```
clientUser.email,  
order.id,  
`${order.vehicle.brand} ${order.vehicle.model}`,  
).catch((err) => console.error('Email send error:', err));  
}
```

Изменение статуса заказа сопровождается рядом дополнительных действий. Помимо обновления поля status, при переводе заказа в состояние COMPLETED автоматически фиксируется дата завершения в поле completedAt. Каждое изменение статуса регистрируется в таблице аудита, а клиент получает уведомление с названием нового статуса. Email-уведомление формируется по HTML-шаблону с цветовой индикацией, визуально отражающей текущий этап выполнения заказа.

Операция добавления запчасти к заказу начинается с проверки складского остатка: контроллер убеждается, что запрошенное количество не превышает доступное. При успешной проверке создаётся запись в таблице OrderPart, при этом цена фиксируется на момент использования, что исключает влияние последующих изменений прайс-листа на уже оформленные заказы. Остаток на складе уменьшается атомарной операцией декремента, после чего вызывается функция пересчёта итоговой стоимости заказа. Последняя загружает все привязанные услуги и запчасти, вычисляет их стоимость по формуле:

$$\text{totalPrice} = \Sigma(\text{service.price} \times \text{service.quantity}) + \Sigma(\text{part.price} \times \text{part.quantity})$$

и сохраняет в поле totalPrice.

Удаление заказа допускается только для записей со статусом “ACCEPTED” или “CANCELLED”, что предотвращает случайное уничтожение данных по заказам, находящимся в работе. Перед удалением контроллер восстанавливает складские остатки – для каждой привязанной запчасти количество инкрементируется на использованный объём. Затем с диска удаляются файлы фотографий, после чего последовательно удаляются все связанные записи: фотографии, история статусов, услуги, запчасти и

уведомления. Только после полной очистки зависимостей удаляется сам заказ. Такая последовательность обусловлена ограничениями ссылочной целостности базы данных и исключает возникновение записей, которые будут забивать память. Демонстрация кода находится в Приложении Ё, Листинг 1.

3.4. Реализация складского учёта

Модуль складского учёта обеспечивает актуализацию остатков запчастей за счёт трёх основных операций: списания, прихода и возврата. При выполнении списания (метод “addPartToOrder”) осуществляется последовательная проверка доступности требуемого количества на складе, после чего создаётся запись связи между заказом и запчастью (“OrderPart”), производится уменьшение остатка, а также выполняется пересчёт общей стоимости заказа.

Операция прихода (метод “createSupply”) реализована через создание сущности поставки (Supply) с вложенными позициями (SupplyItem). Для каждой позиции выполняется увеличение остатка соответствующей запчасти, это обеспечивает корректное отражение поступлений на склад.

Возврат запчастей реализуется при удалении заказа (метод “deleteOrder”). В этом случае для каждой связанной позиции “OrderPart” выполняется обратная операция – увеличение остатка, что позволяет восстановить складские данные без потери целостности.

Также дополнительно реализован механизм контроля минимальных остатков, позволяющий выявлять дефицитные позиции. В рамках эндпоинта “getLowStockParts” выполняется выборка запчастей, у которых текущее количество меньше либо равно установленному минимальному порогу. Это обеспечивает своевременное информирование кладовщика и позволяет оперативно формировать заказы поставщикам, снижая риск дефицита и простоя. Демонстрация находится в Приложение Ё, Листинг 2.

3.6. Сервис E-mail уведомлений

Модуль сервисных email-уведомлений реализован в виде отдельного компонента (emailService.ts) и предназначен для отправки HTML-писем через SMTP с использованием библиотеки Nodemailer. Архитектура сервиса предусматривает такую инициализацию транспортного уровня: SMTP-транспорт создаётся только при первом обращении и далее повторно используется, что соответствует паттерну Singleton и снижает накладные расходы на повторное подключение.

Формирование писем осуществляется с использованием HTML-шаблонов, включающих встроенные стили. В шаблонах применяется единая цветовая схема, а также визуальная индикация статуса заказа, что повышает читаемость уведомлений. Для обеспечения корректного отображения в почтовых клиентах используется адаптивная вёрстка с ограничением ширины и inline-стилями. Демонстрация находится в Приложение Ё, Листинг 3.

Отправка уведомлений реализована в асинхронном режиме. Методы отправки (например, уведомления о создании заказа и изменении его статуса) вызываются в контроллерах с обработкой ошибок через .catch(), что исключает блокировку формирования HTTP-ответа при задержках или сбоях SMTP-сервера.

3.7. Реализация подсистемы защиты персональных данных

Подсистема защиты персональных данных реализована на уровне приложения и включает совокупность механизмов, направленных на обеспечение конфиденциальности, целостности и контролируемого доступа к данным пользователей.

Хранение паролей осуществляется с использованием алгоритма хеширования bcrypt с коэффициентом сложности 10. Это обеспечивает генерацию уникальной соли и исключает возможность восстановления исходного пароля.

Для предотвращения утечки чувствительной информации реализована фильтрация полей в ответах API. Во всех контроллерах применяется явная проекция данных, при которой в ответ включаются только безопасные атрибуты пользователя, а поле пароля полностью исключается.

Листинг 3.7.1 – Фильтрация полей в ответах API

```
const user = await prisma.user.findUnique({
  where: { id: userId },
  select: { id: true, email: true, firstName: true, lastName: true, phone:
true, role: true },
});
```

Аутентификация пользователей построена на основе JWT-токенов. Токен передаётся в заголовке запроса, проходит проверку подписи и используется для извлечения идентификатора пользователя и его роли. Срок действия токена ограничен, что снижает риски несанкционированного доступа. Дополнительно реализовано ролевое разграничение доступа: каждый маршрут защищён middleware, проверяющим соответствие роли пользователя установленным требованиям.

Листинг 3.7.2 – Ролевое разграничение доступа

```
export const authorize = (...roles: Role[]) => {
  return (req: Request, res: Response, next: NextFunction): void => {
    if (!roles.includes(req.user.role)) {
      res.status(403).json({ error: 'Нет доступа' });
      return;
    }
    next();
  };
};
```

Безопасность взаимодействия усиливается механизмом ограничения частоты запросов на маршрутах аутентификации, что предотвращает перебор паролей. При превышении установленного лимита сервер возвращает соответствующий код ошибки (демонстрация кода Приложение Ё, Листинг 4). Также в приложении используются стандартные HTTP-заголовки

безопасности, автоматически устанавливаемые middleware, что обеспечивает защиту от распространённых веб-угроз, включая XSS и clickjacking.

Также стоит упомянуть, что дополнительно реализован механизм автоматического завершения сессии при получении ошибки авторизации, а также функция анонимизации персональных данных по запросу пользователя. В рамках данной операции персональные сведения заменяются обезличенными значениями, учётная запись деактивируется, при этом история заказов и связанные данные сохраняются без возможности идентификации пользователя. Реализован эндпоинт “DELETE /api/profile/account”, доступный аутентифицированному пользователю. Для предотвращения случайного удаления требуется подтверждение текущим паролем. Операция выполняется в рамках транзакции Prisma (‘prisma.\$transaction’), обеспечивающей атомарность:

1. ФИО заменяется на «Удалённый Пользователь».
2. Email заменяется на уникальный технический адрес “deleted_<hex>@anonymous.local” (hex – 4 случайных байта через “crypto.randomBytes”), что исключает конфликт уникальности с другими записями.
3. Телефон обнуляется.
4. Пароль заменяется криптографически случайной строкой длиной 32 байта (не является валидным bcrypt-хешем, что делает вход невозможным).
5. Учётная запись деактивируется (“isActive: false”).
6. VIN-номера и государственные регистрационные знаки всех автомобилей клиента обнуляются.
7. Уведомления клиента удаляются.

3.8. Реализация клиентской части

Клиентская часть системы реализована на базе «React 19» с использованием «TypeScript». Структура приложения включает страничные

компоненты, глобальные контексты состояния и переиспользуемые UI-компоненты.

Маршрутизация приложения разделена на публичные и защищённые маршруты (таблица маршрутов представлена в Приложении Ё, Листинг 5.). Доступ к защищённым разделам осуществляется через специализированные компоненты, выполняющие проверку аутентификации и роли пользователя. Дополнительно реализован механизм автоматического перенаправления в зависимости от роли, а также единый каркас интерфейса с общей навигацией и областью отображения содержимого.

Глобальное состояние управляется с использованием «React Context». Контекст аутентификации обеспечивает хранение данных пользователя, управление сессией и взаимодействие с локальным хранилищем браузера.

Для взаимодействия с серверной частью используется настроенный HTTP-клиент на базе Axios. Он автоматически добавляет токен авторизации к каждому запросу и обрабатывает ошибки аутентификации, обеспечивая корректное завершение пользовательской сессии при истечении срока действия токена (демонстрация находится в Приложении Ё, Листинг 6).

Интерфейс приложения построен на основе компонентов каркаса, включающих верхнюю и боковую панели навигации, адаптированные под роль пользователя и тип устройства (продемонстрировано в Приложение Ё, Рисунок 3, Рисунок 4). Основной функционал реализован в виде набора страничных компонентов, охватывающих модули аутентификации, управления заказами, складского учёта, отчётности и работы с клиентами. Визуализация данных, включая аналитические отчёты, выполнена с использованием графических библиотек.

3.9. Тестирование системы

Тестирование информационной системы построена на автоматизированных проверках серверного слоя, потому что он наиболее важен с точки зрения корректности данных и безопасности. Интеграционный

тест, обращающийся к реальному экземпляру PostgreSQL через HTTP-эндпоинт, одновременно проверяет маршрутизацию, middleware-аутентификацию, валидацию прав, выполнение SQL-запросов через Prisma и формат ответа – то есть всю цепочку обработки запроса.

В рамках данной работы можно выделить следующие уровни тестирования:

- Интеграционные автотесты серверной части – это основной уровень покрытия. Каждый тест выполняет реальный HTTP-запрос к Express-приложению и проверяет как код ответ, так и структуру возвращаемых данных и состояние базы данных.
- Ручное функциональное тестирование клиентской части – это проверка пользовательских сценариев в браузере на тестовом наборе seed-данных.
- Ручное тестирование безопасности – это проверка защитных механизмов.

В качестве фреймворка для тестирования был выбран Jest 30 с плагином ts-jest, а для тестирования HTTP-эндпоинтов – библиотека supertest 7, которая позволяет выполнять запросы к экземпляру Express-приложения без запуска реального сервера на порту.

Набор автотестов включает 5 тест-файлов с общим количеством 42 тест-кейса, которые распределены по функциональным областям. Таблица с тестами находится в Приложении Ё, Таблица 1.

Список использованных литературных источников

1. Автостат-Инфо. Новости [Электронный ресурс]. – URL: <https://avtostat-info.com/News/12353> (дата обращения: 10.03.2026).
2. Автостат. Инфографика [Электронный ресурс]. – URL: <https://www.autostat.ru/infographics/61565/> (дата обращения: 10.03.2026).
3. Автостат. Инфографика [Электронный ресурс]. – URL: <https://www.autostat.ru/infographics/61553/> (дата обращения: 10.03.2026).
4. Долгополук Эрика Эриковна, Селезнев Егор Сергеевич Роботизация и автоматизация малого и среднего предпринимательства // Вестник экономики и права. 2025. №99. URL: <https://cyberleninka.ru/article/n/robotizatsiya-i-avtomatizatsiya-malogo-i-srednego-predprinimatelstva> (дата обращения: 15.03.2026).
5. Altera Invest. Автосервис с нуля: бизнес без компромиссов [Электронный ресурс]. – URL: <https://alterainvest.ru/rus/blogi/avtoservis-s-nulya-biznes-bez-kompromissov/> (дата обращения: 15.03.2026).
6. АвтоДилер. О компании [Электронный ресурс]. – URL: <https://autodealer.ru/about> (дата обращения: 15.03.2026).
7. PickTech. АвтоДилер Онлайн [Электронный ресурс]. – URL: <https://picktech.ru/product/avtodiler-onlayn/> (дата обращения: 29.04.2026).
8. 1С:Предприятие 8. Автосервис: возможности [Электронный ресурс]. – URL: <https://solutions.1c.ru/catalog/autoservice/features> (дата обращения: 24.03.2026).
9. 1С-Рарус. 1С:Автосервис [Электронный ресурс]. – URL: <https://rarus.ru/1c-auto/1c8-avtoservis/> (дата обращения: 29.03.2026).
10. STOCRM. Программа CRM для автосервиса [Электронный ресурс]. – URL: <https://stocrm.ru/> (дата обращения: 29.03.2026).
11. Startpack. STOCRM – обзор сервиса [Электронный ресурс]. – URL: <https://startpack.ru/application/stocrm> (дата обращения: 29.03.2026).

12. Splus. Онлайн-программа для учёта в автосервисе [Электронный ресурс]. – URL: <https://splus.ru/> (дата обращения: 30.03.2026).
13. Splus. Программа для автосервисов и автомоек [Электронный ресурс]. – URL: <https://splus.ru/crm/auto/autorepair> (дата обращения: 30.03.2026).
14. Соболев В. В., Мосейчук А. А. Автосервис с позиции теории систем массового обслуживания // Решетневские чтения. 2018. №. URL: <https://cyberleninka.ru/article/n/avtoservis-s-pozitsii-teorii-sistem-massovogo-obsluzhivaniya> (дата обращения: 30.03.2026).
15. Prisma. Prisma Documentation [Электронный ресурс]. – URL: <https://www.prisma.io/docs> (дата обращения: 31.03.2026).
16. Express.js. Express – Node.js web application framework [Электронный ресурс]. – URL: <https://expressjs.com/> (дата обращения: 31.03.2026).
17. React. Learn React [Электронный ресурс]. – URL: <https://react.dev/learn> (дата обращения: 31.03.2026).
18. Федеральный закон "О персональных данных" от 27.07.2006 N 152-ФЗ (последняя редакция) // КонсультантПлюс URL: https://www.consultant.ru/document/cons_doc_LAW_61801/ (дата обращения: 29.03.2026).

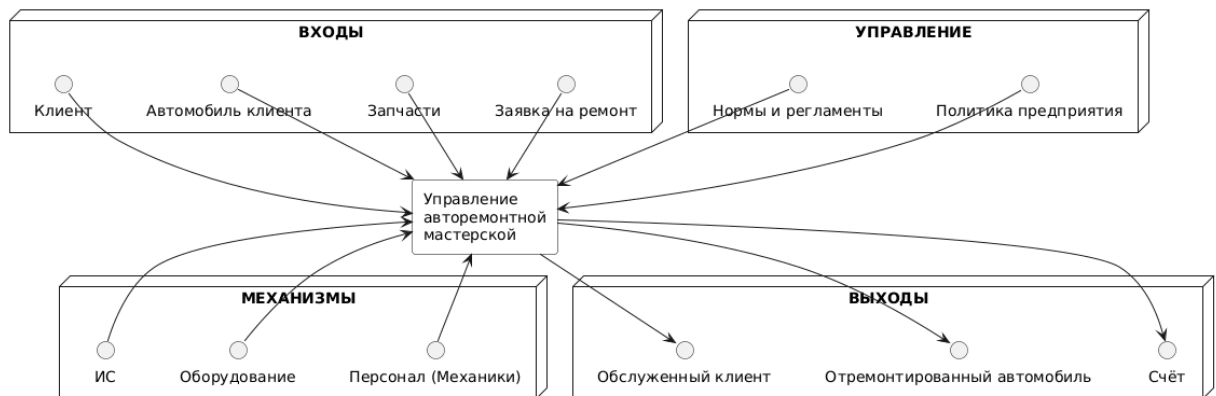
Приложения

Приложение А - Сравнительная таблица существующих решений

Критерий	АвтоДилер Онлайн	1С:Автосервис	Splus	STOCRM
Заказ-наряды	Да	Да	Да	Да
Складской учёт запчастей	Да	Да	Да	Да
Нормы времени работ	Да	Да	Да	Да
Каталоги запчастей / оценка	Да	Да	Да	Да
CRM / клиентская база	Да	Частично	Да	Да
Онлайн-запись клиентов	Да	Нет	Да	Да
Уведомления (SMS / Email / мессенджеры)	Да	Нет	Да	Да
Мобильное приложение	Да	Нет	Да	Да
Расчёт зарплаты	Да	Да	Да	Да
Воронка продаж	Нет	Нет	Нет	Да
Канбан-доска управления процессами	Нет	Нет	Нет	Да
Аналитика рентабельности	Частично	Да	Частично	Да
Интеграция с бухгалтерией (1С)	Да	Да	Да	Да
Облачный доступ (SaaS)	Да	Нет	Да	Да

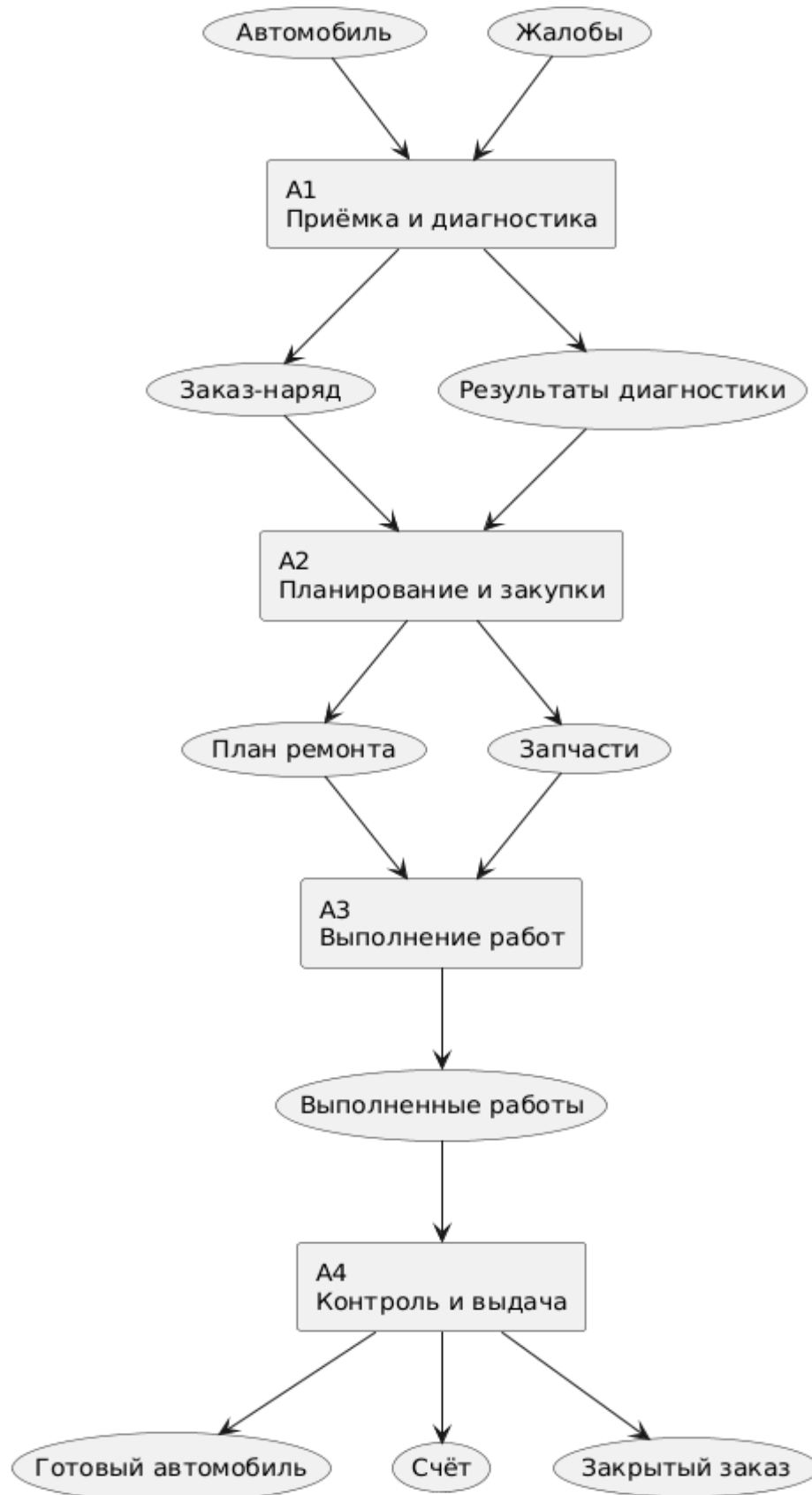
Приложение Б - Диаграммы бизнес-процессов

IDEF0 A-0: Управление авторемонтной мастерской

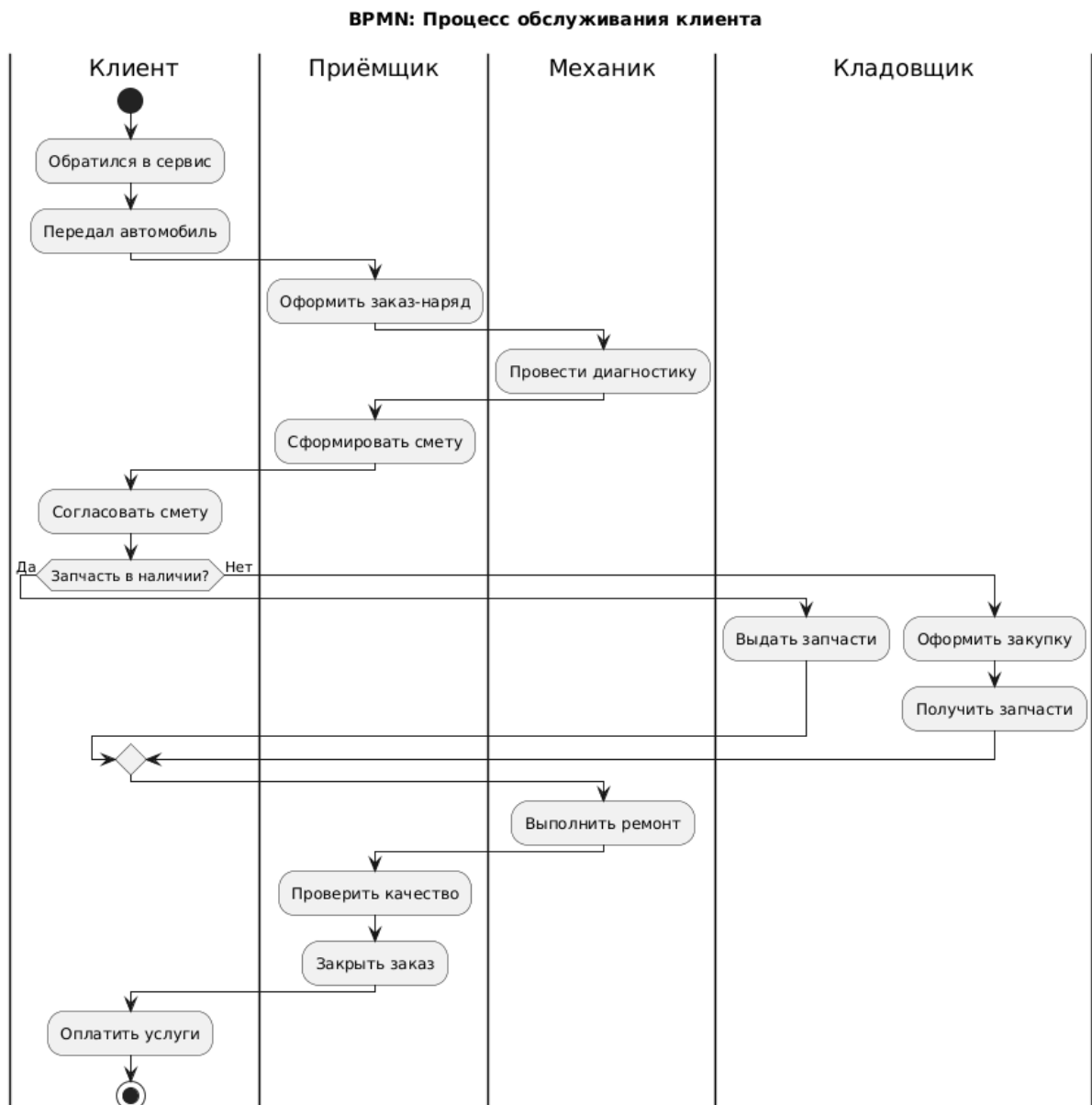


Приложение 2.1. Управление авторемонтной мастерской.

IDEF0 A0: Декомпозиция процесса



Приложение 2.2. Декомпозиция процесса.



Приложение 2.3. Процесс обслуживания клиента.

Приложение В – Бриф проекта БРИФ ПРОЕКТА

Информационная система управления авторемонтной мастерской

1. Общие сведения о проекте	
Название проекта	Разработка информационной системы управления средней авторемонтной мастерской
Автор (студент)	Дмитрий Сатюков
Дата составления	15 марта 2026 г.
2. Описание проекта	

Краткое описание	Веб-приложение, которое полностью автоматизирует повседневную работу небольшой независимой авторемонтной мастерской. Доступ с компьютера, планшета и смартфона – без установки ПО.
3. Проблема, которую решает проект	
Ключевые проблемы отрасли	• Ошибки в учёте запчастей • Простои механиков из-за плохого планирования • Потеря выручки из-за неучтённых работ и человеческого фактора • Отсутствие фотоотчётов, точных сроков и напоминаний клиентам • Отсутствие реальной аналитики у владельца
4. Цель и задачи проекта	
Цель проекта	Разработка и реализация информационной системы для автоматизации процессов управления средней авторемонтной мастерской, обеспечивающей повышение эффективности учёта заказов, управления ресурсами и взаимодействия с клиентами.
Задачи проекта	1. Сформировать требования к информационной системе управления авторемонтной мастерской. 2. Спроектировать архитектуру информационной системы и базу данных с учётом специфики предметной области. 3. Разработать модуль учёта клиентов и транспортных средств с хранением истории обращений и ремонтов. 4. Реализовать модули заказ-нарядов и складского учёта запчастей, включая формирование заказов на ремонт, составить тест-кейсы. 5. Провести тестирование системы и дать рекомендации по развертыванию и сопровождению.
5. Целевая аудитория и пользователи	
Роли пользователей	• Владелец / директор – полный доступ к аналитике и отчётам • Приёмщики заказов • Механики – мобильный доступ • Кладовщик • Бухгалтер – частичный доступ
Целевой сегмент	Небольшие независимые мастерские без штатного IT-специалиста и бюджета на дорогие решения типа 1С.
6. Основные требования к системе	
Архитектура и доступ	• Облачная архитектура – доступ с любого устройства без установки • Современный интуитивный интерфейс с мобильной адаптивностью
Основные модули	• CRM и клиентская база • Заказ-наряды • Складской учёт • Планирование и календарь • Финансы и аналитика • Уведомления (Email)
Интеграции	• Простая интеграция с 1С (выгрузка)

Безопасность и запуск	• Рольевая модель доступа • Резервное копирование
7. Ожидаемые результаты	
Результаты проекта	• Готовая работающая система • Сокращение времени оформления заказа-наряда • Минимизация ошибок на складе и простоев • Повышение лояльности клиентов за счёт прозрачности и уведомлений

Приложение Г – Use Case Диаграммы

Use Case Diagram — Клиент (Client)



Рисунок 4.1. Use Case диаграмма клиента

Use Case Diagram — Мастер (Master)



Рисунок 4.2. Use Case Диаграмма мастера

Use Case — Кладовщик



Рисунок 4.3. Use Case Диаграмма кладовщика.

Use Case — Менеджер

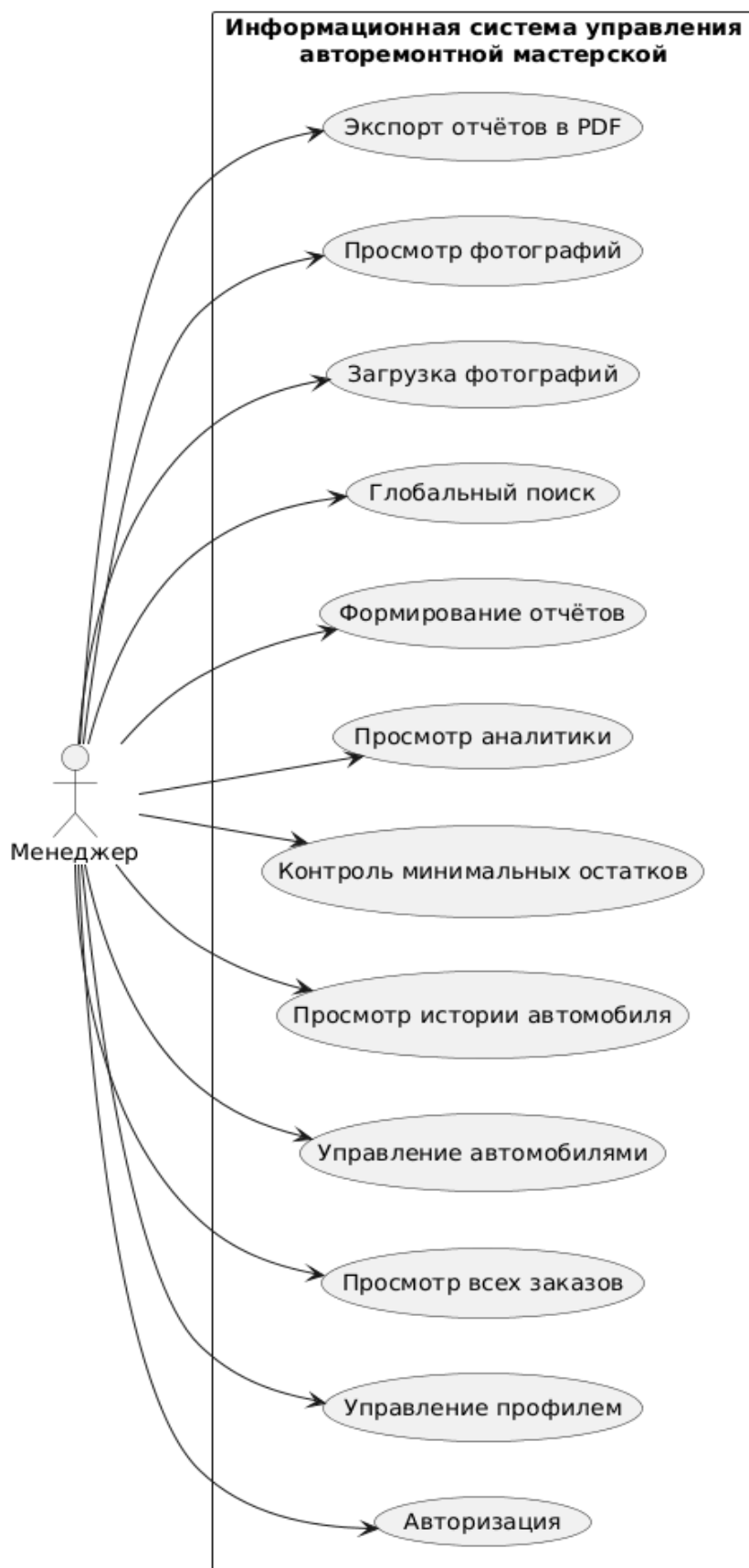


Рисунок 4.4. Use Case диаграмма менеджера.

Приложение Д – Интерфейс. Проектирование

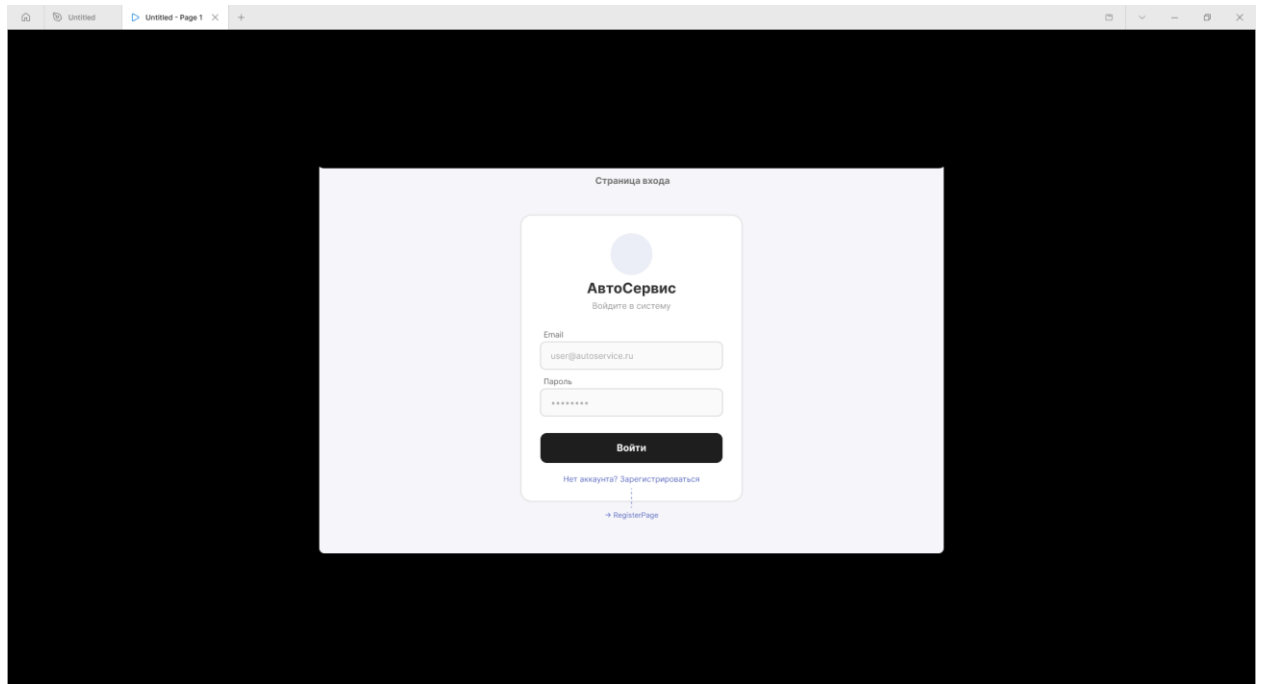


Рисунок 1.1 – Экран авторизации

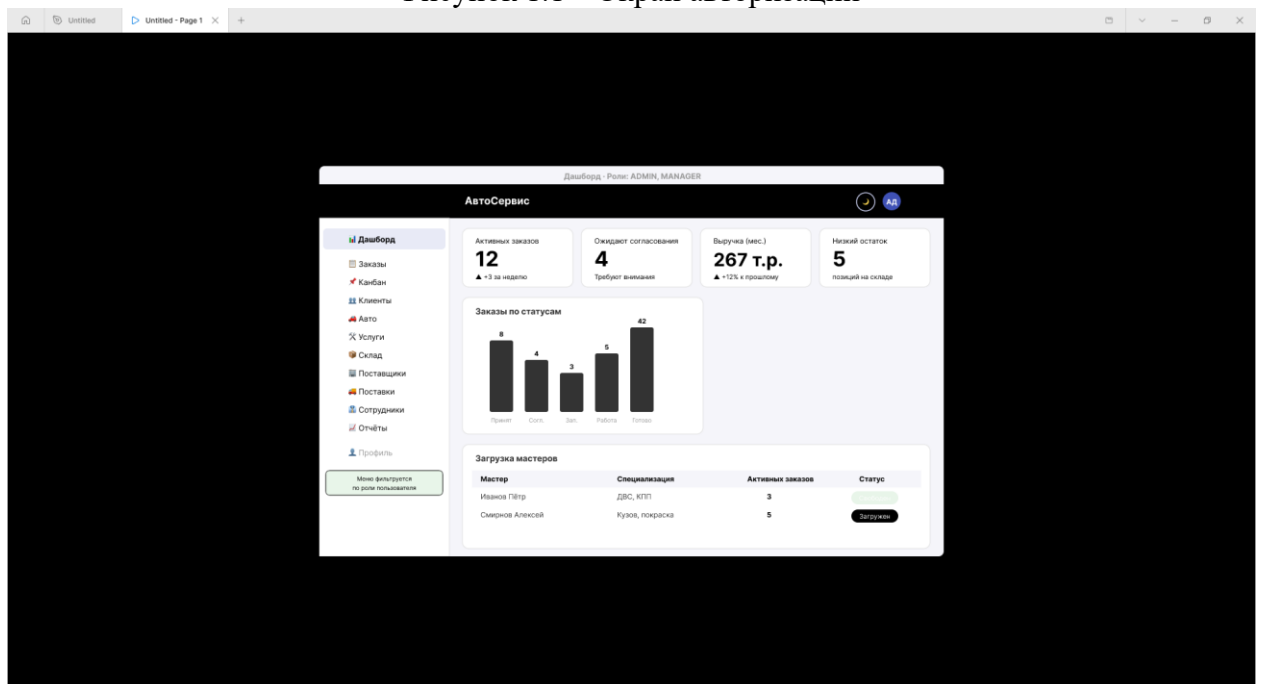


Рисунок 1.2 – Главная страница или дашборд

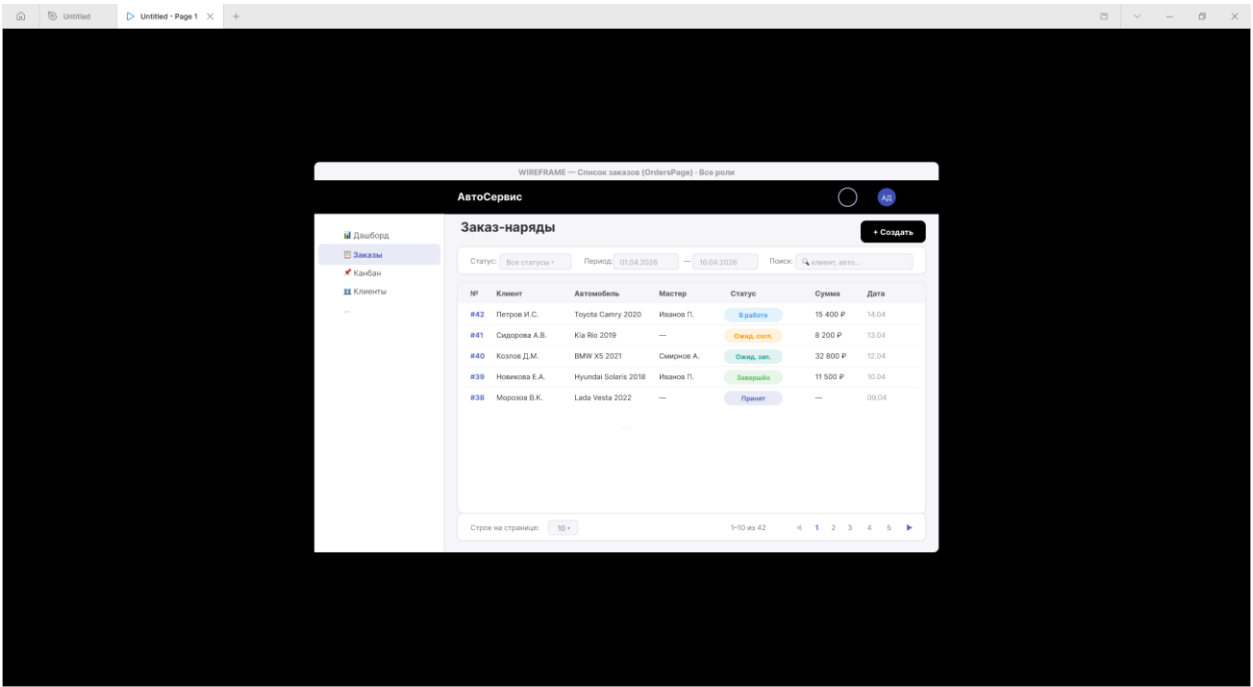


Рисунок 1.3 – Экран заказ-нарядов

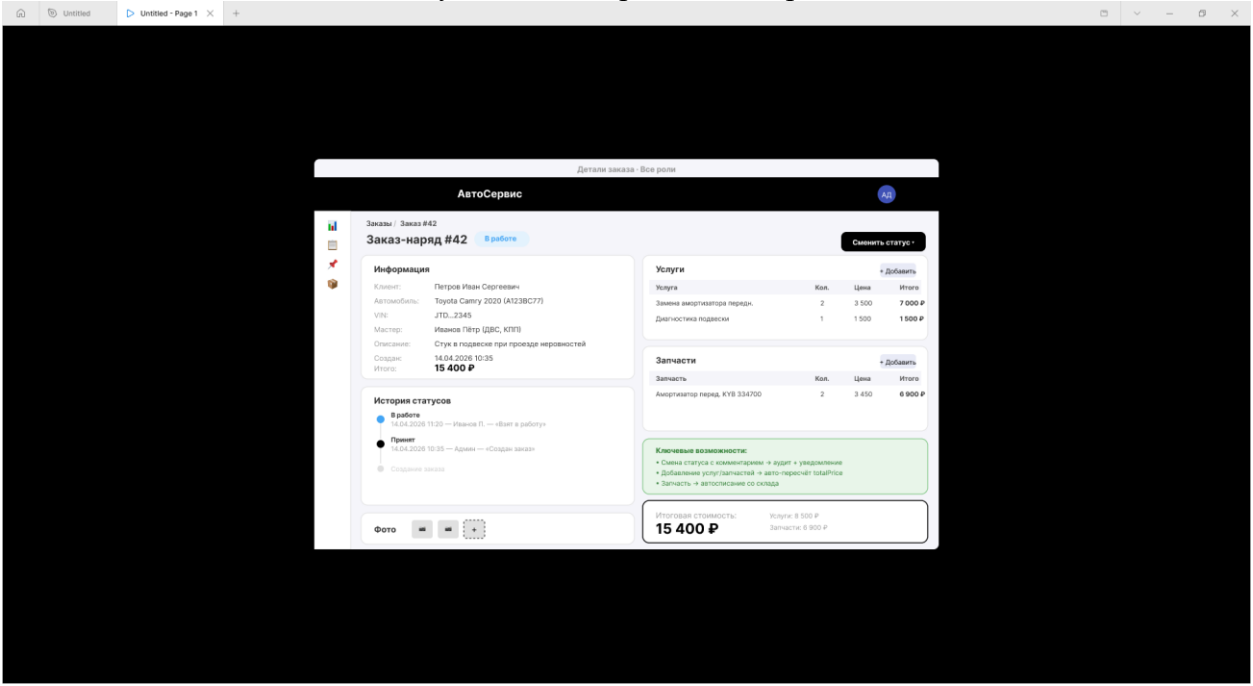


Рисунок 1.4. – Экран определённого заказ-наряда

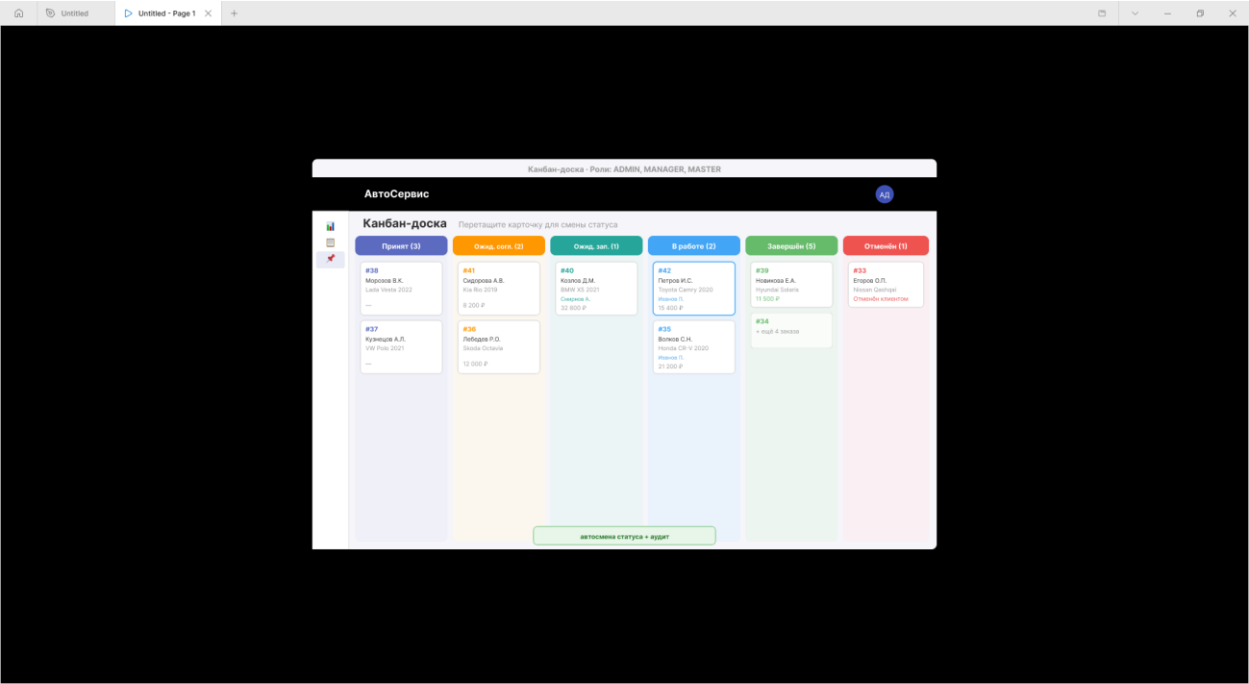


Рисунок 1.5. – Канбан-доска.

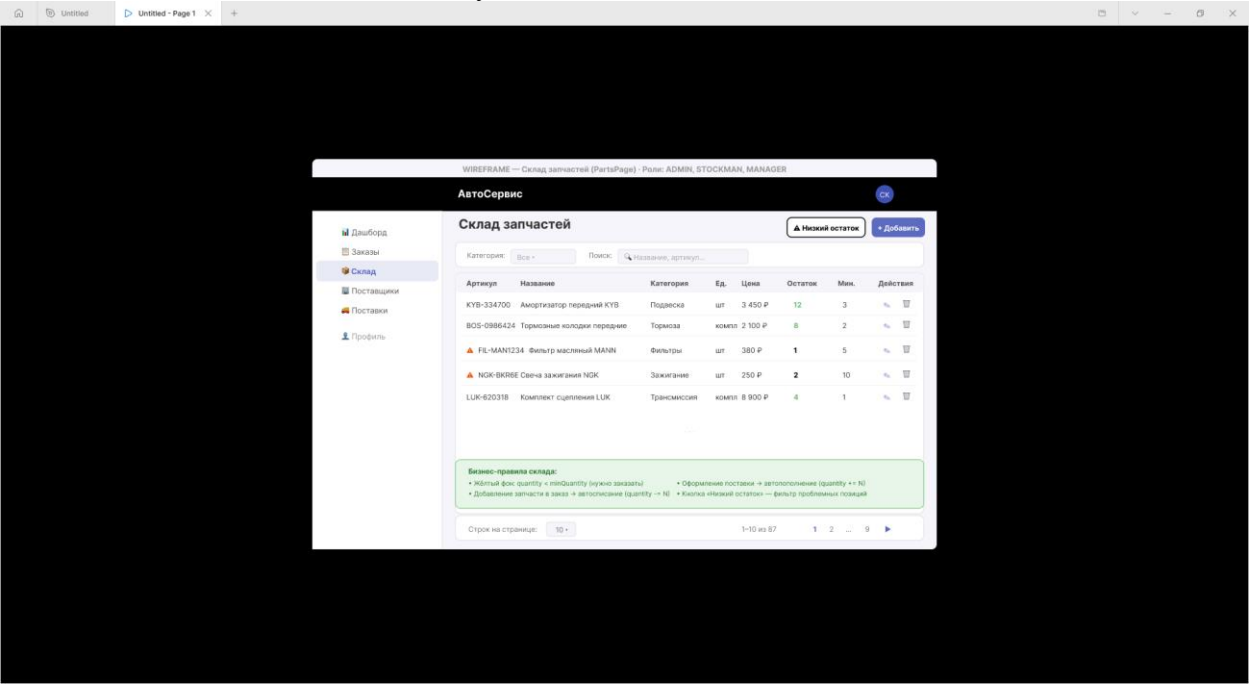


Рисунок 1.6. – Склад запчастей

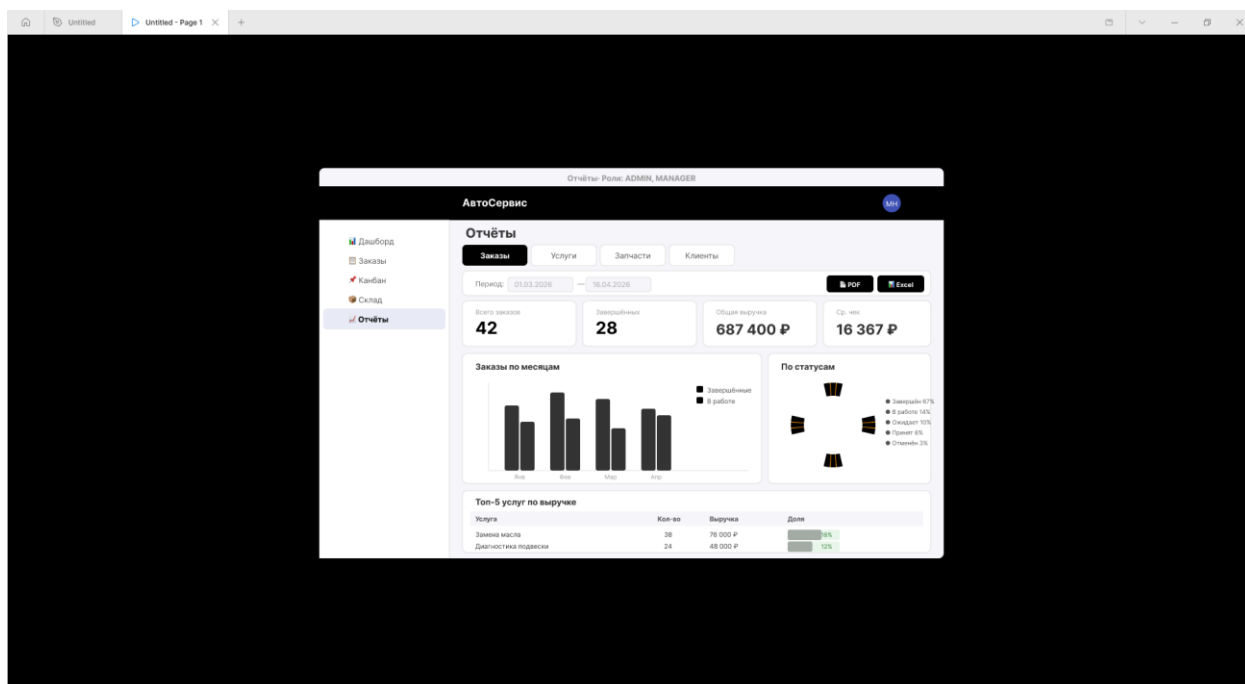


Рисунок 1.7. – Экран отчётности

Приложение Е – Таблицы сравнений технологий

Критерий	Node.js (Express)	Python (Django)	Java (Spring Boot)
Скорость разработки	Высокая	Высокая	Средняя
Единый язык для клиентской и серверной частей	Да	Нет	Нет
Поддержка асинхронной обработки запросов	Встроенная	Ограниченная	Реализуется через пул потоков
Развитость экосистемы библиотек и пакетов	Более 2 млн	Около 400 тыс.	Около 500 тыс.
Трудоёмкость начальной реализации	Низкая	Средняя	Высокая
Поддержка типизации	TypeScript	Муру	Нативная

Таблица 5.1. Сравнение серверных платформ.

Критерий	PostgreSQL	MySQL	MongoDB
Тип модели данных	Реляционная	Реляционная	Документно-ориентированная
Поддержка сложных аналитических и связанных запросов	Полная	Частично ограниченная	Средняя
Обеспечение целостности данных	Высокое	Высокое	Ограниченное

Поддержка транзакций	ACID-полная	ACID-полная	Ограниченная
Соответствие структуре предметной области	Высокое	Высокое	Низкое

Таблица 5.2. Сравнение систем управления базами данных.

Критерий	Prisma	TypeORM	Sequelize
Уровень типобезопасности	Высокий (полный)	Средний (частичный)	Низкий
Поддержка декларативного описания схемы данных	Да	Нет	Нет
Поддержка автоматизированных миграций	Да	Частично	Частично
Возможность интроспекции существующей базы данных	Да	Ограниченно	Ограниченно
Поддержка TypeScript	Нативная	Хорошая	Ограниченная

Таблица 5.3. Сравнение ORM-библиотек.

Критерий	React	Vue	Angular
Архитектурная модель	UI-библиотека	Прогрессивный фреймворк	Полноценный фреймворк
Развитость экосистемы	Очень высокая	Высокая	Высокая
Сложность освоения и внедрения	Средняя	Низкая	Высокая
Поддержка TypeScript	Полная	Полная	Нативная
Гибкость выбора дополнительных библиотек	Максимальная	Высокая	Ограниченная

Таблица 5.4. Сравнение фреймворков клиентского приложения.

Приложение Ё – Модели данных

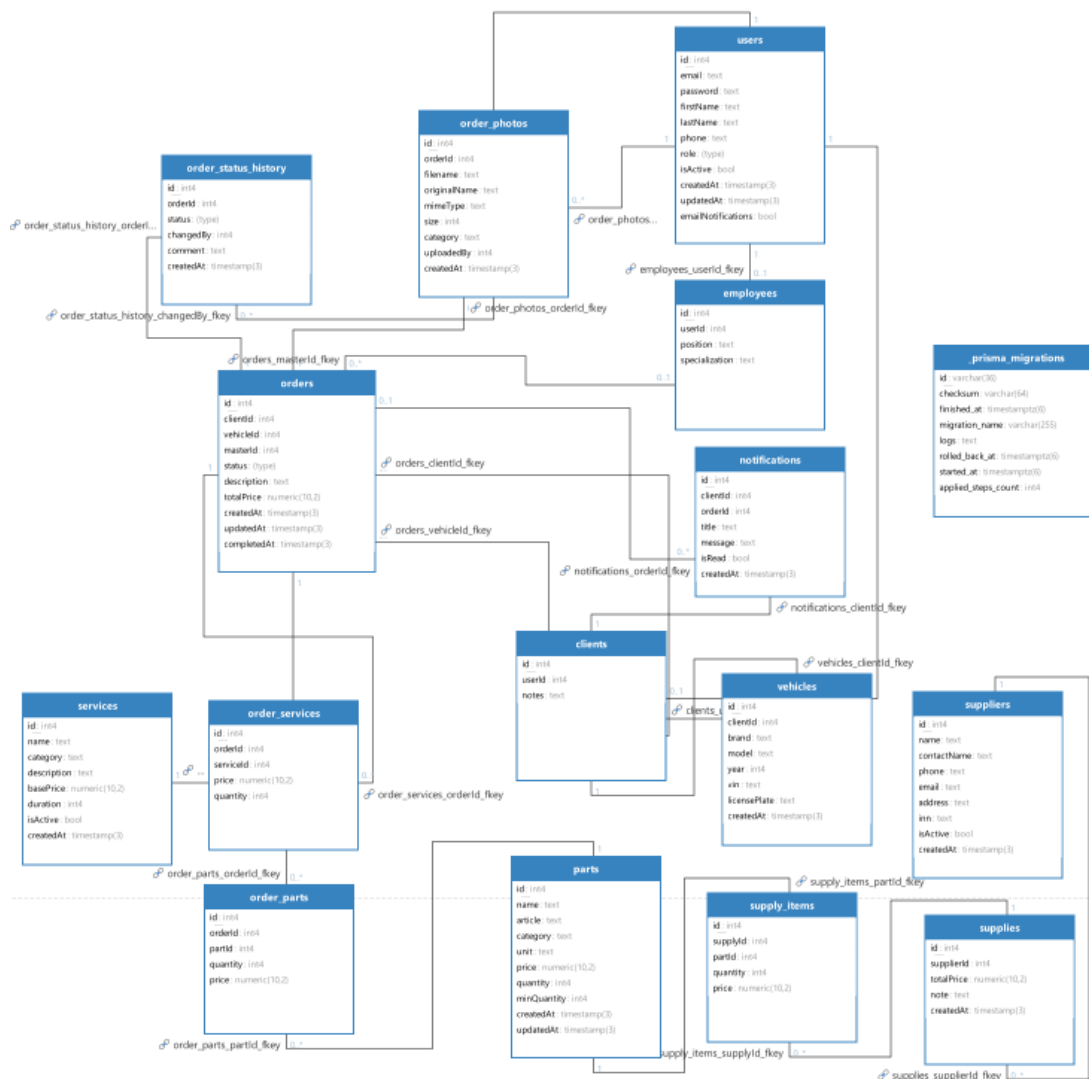


Рисунок 6.1. ER-диаграмма

Приложение Ж – Реализация и демонстрация кода

Листинг 1 – Демонстрация ключевых функций *orderController.ts*

```
export const createOrder = async (req: Request, res: Response):
Promise<void> => {
  try {
    const { clientId, vehicleId, masterId, description } = req.body;

    const order = await prisma.order.create({
      data: {
        clientId,
        vehicleId,
        masterId,
        description,
        status: OrderStatus.ACCEPTED,
        statusHistory: {
          create: {
            status: OrderStatus.ACCEPTED,
            changedBy: req.user!.userId,
            comment: 'Заказ создан',
          },
        },
      },
    });
  } catch (error) {
    console.error('Error creating order:', error);
    res.status(500).json({ message: 'Internal server error' });
  }
}
```

```

    },
  },
  include: {
    client: { include: { user: { select: { firstName: true, lastName:
true } } } },
    vehicle: true,
  },
});

await prisma.notification.create({
  data: {
    clientId,
    orderId: order.id,
    title: 'Новый заказ',
    message: `Создан заказ-наряд #${order.id} для вашего автомобиля
${order.vehicle.brand} ${order.vehicle.model}`,
  },
});

const clientUser = await prisma.user.findFirst({
  where: { client: { id: clientId } },
  select: { email: true, emailNotifications: true },
});
if (clientUser?.emailNotifications) {
  sendNewOrderEmail(
    clientUser.email,
    order.id,
    `${order.vehicle.brand} ${order.vehicle.model}`,
  ).catch((err) => console.error('Email send error:', err));
}

res.status(201).json(order);
} catch (error) {
  console.error('CreateOrder error:', error);
  res.status(500).json({ error: 'Ошибка создания заказа' });
}
};

export const deleteOrder = async (req: Request, res: Response):
Promise<void> => {
  try {
    const id = Number(req.params.id);
    const order = await prisma.order.findUnique({ where: { id } });
    if (!order) {
      res.status(404).json({ error: 'Заказ не найден' });
      return;
    }
    if (order.status !== 'ACCEPTED' && order.status !== 'CANCELLED') {
      res.status(400).json({ error: 'Можно удалить только заказ в статусе
"Принят" или "Отменён"' });
      return;
    }
    // Return parts to stock
    const orderParts = await prisma.orderPart.findMany({ where: { orderId:
id } });
    for (const op of orderParts) {
      await prisma.part.update({
        where: { id: op.partId },
        data: { quantity: { increment: op.quantity } },
      });
    }
    // Delete photos from disk

```

```

const photos = await prisma.orderPhoto.findMany({ where: { orderId: id }
});
for (const photo of photos) {
  const filePath = path.join(__dirname, '../../uploads/orders',
photo.filename);
  if (fs.existsSync(filePath)) {
    fs.unlinkSync(filePath);
  }
}

// Delete related records
await prisma.orderPhoto.deleteMany({ where: { orderId: id } });
await prisma.orderStatusHistory.deleteMany({ where: { orderId: id } });
await prisma.orderService.deleteMany({ where: { orderId: id } });
await prisma.orderPart.deleteMany({ where: { orderId: id } });
await prisma.notification.deleteMany({ where: { orderId: id } });
await prisma.order.delete({ where: { id } });
res.json({ success: true });
} catch (error) {
  console.error('DeleteOrder error:', error);
  res.status(500).json({ error: 'Ошибка удаления заказа' });
}
};

```

#	Клиент	Автомобиль	Мастер	Статус	Сумма	Дата
31	Светлана Белова	Volvo XC60	—	Ожидает согласования	—	01.04.2026
29	Елена Новикова	Mazda CX-5	—	Принят	11 500 Р	01.04.2026
28	Мария Сидорова	Kia Rio	—	Принят	4 700 Р	01.04.2026
27	Андрей Петров	Toyota Camry	—	Принят	—	01.04.2026
26	Денис Захаров	Mitsubishi Outlander	Алексей Мастеров	Ожидает согласования	—	01.04.2026
30	Максим Лебедев	Ford Focus	—	Завершен	10 200 Р	31.03.2026
25	Ирина Федорова	Mercedes-Benz C-Class	Дмитрий Слесарев	Ожидает запчастей	—	31.03.2026
24	Татьяна Соколова	Renault Duster	Николай Кузнецов	Ожидает запчастей	—	31.03.2026
18	Светлана Белова	Volvo XC60	Дмитрий Слесарев	В работе	16 800 Р	31.03.2026
16	Виктор Морозов	Lada Vesta	Николай Кузнецов	В работе	5 000 Р	31.03.2026
23	Анна Волкова	Nissan Qashqai	Алексей Мастеров	Ожидает согласования	—	30.03.2026

Рисунок 1 – Демонстрация страницы заказ-нарядов

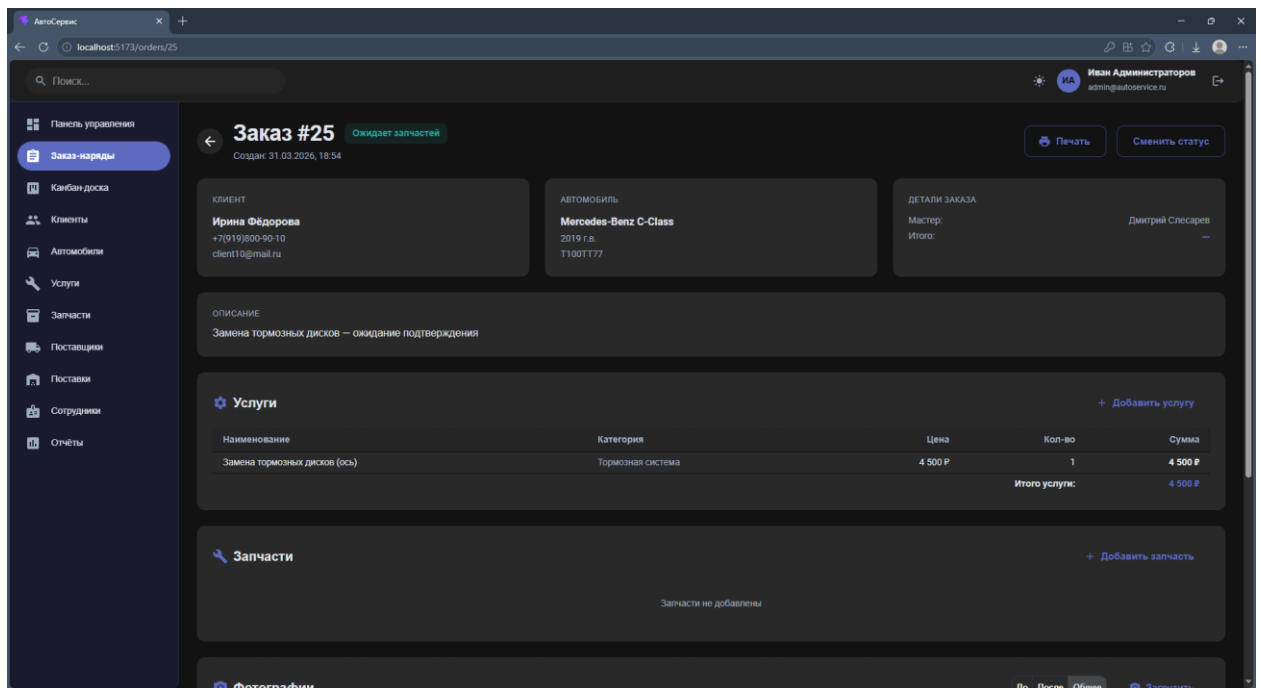


Рисунок 2 – Пример страницы заказ-наряда

Листинг 2 – Фрагмент кода. Механизм контроля остатков

```
export const getLowStockParts = async (_req: Request, res: Response):
Promise<void> => {
  try {
    const parts = await prisma.part.findMany({
      where: {
        quantity: { lte: prisma.part.fields.minQuantity },
      },
    });
    res.json(parts);
  } catch (error) {
    // Fallback: получить все и отфильтровать на уровне приложения
    const allParts = await prisma.part.findMany();
    const lowStock = allParts.filter(p => p.quantity <= p.minQuantity);
    res.json(lowStock);
  }
};
```

Листинг 3 – Полный код emailService

```
import nodemailer from 'nodemailer';

let transporter: nodemailer.Transporter | null = null;

function getTransporter(): nodemailer.Transporter | null {
  if (!process.env.SMTP_HOST) return null;

  if (!transporter) {
    transporter = nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port: Number(process.env.SMTP_PORT) || 587,
      secure: Number(process.env.SMTP_PORT) === 465,
      auth: {
        user: process.env.SMTP_USER,
        pass: process.env.SMTP_PASS,
      },
    });
  }
}

return transporter;
```



```

}

const STATUS_LABELS: Record<string, string> = {
  ACCEPTED: 'Принят',
  AWAITING_APPROVAL: 'Ожидает согласования',
  AWAITING_PARTS: 'Ожидает запчастей',
  IN_PROGRESS: 'В работе',
  COMPLETED: 'Завершён',
  CANCELLED: 'Отменён',
};

const STATUS_COLORS: Record<string, string> = {
  ACCEPTED: '#5c6bc0',
  AWAITING_APPROVAL: '#ff9800',
  AWAITING_PARTS: '#26a69a',
  IN_PROGRESS: '#42a5f5',
  COMPLETED: '#66bb6a',
  CANCELLED: '#ef5350',
};

export async function sendOrderStatusEmail(
  toEmail: string,
  orderId: number,
  status: string,
  vehicleInfo: string,
): Promise<void> {
  const transport = getTransporter();
  if (!transport) return;

  const statusLabel = STATUS_LABELS[status] || status;
  const statusColor = STATUS_COLORS[status] || '#9e9e9e';
  const fromAddress = process.env.SMTP_FROM || '"АвтоСервис"
<noreply@autoservice.ru>';

  const html = `
    <div style="font-family: 'Inter', Arial, sans-serif; max-width: 600px;
margin: 0 auto; padding: 20px;">
      <div style="background: #5c6bc0; color: white; padding: 20px; border-
radius: 12px 12px 0 0; text-align: center;">
        <h1 style="margin: 0; font-size: 20px;">АвтоСервис</h1>
      </div>
      <div style="background: #ffffff; padding: 24px; border: 1px solid
#e0e0e0; border-top: none; border-radius: 0 0 12px 12px;">
        <h2 style="margin: 0 0 16px; font-size: 18px; color:
#333;">Обновление статуса заказа #${orderId}</h2>
        <p style="color: #666; margin: 0 0 12px;">Автомобиль:
<strong>${vehicleInfo}</strong></p>
        <div style="background: ${statusColor}15; border-left: 4px solid
${statusColor}; padding: 12px 16px; border-radius: 0 8px 8px 0; margin: 16px
0;">
          <p style="margin: 0; color: #333;">Новый статус: <strong
style="color: ${statusColor};">${statusLabel}</strong></p>
        </div>
        <hr style="border: none; border-top: 1px solid #eee; margin: 20px
0;" />
        <p style="color: #999; font-size: 13px; margin: 0;">Это
автоматическое уведомление. Не отвечайте на это письмо.</p>
      </div>
    `;

  await transport.sendMail({

```

```

    from: fromAddress,
    to: toEmail,
    subject: `Заказ #${orderId} - ${statusLabel}`,
    html,
  });
}

export async function sendNewOrderEmail(
  toEmail: string,
  orderId: number,
  vehicleInfo: string,
): Promise<void> {
  const transport = getTransporter();
  if (!transport) return;

  const fromAddress = process.env.SMTP_FROM || '"АвтоСервис"
<noreply@autoservice.ru>';

  const html = `
    <div style="font-family: 'Inter', Arial, sans-serif; max-width: 600px;
margin: 0 auto; padding: 20px;">
      <div style="background: #5c6bc0; color: white; padding: 20px; border-
radius: 12px 12px 0 0; text-align: center;">
        <h1 style="margin: 0; font-size: 20px;">АвтоСервис</h1>
      </div>
      <div style="background: #ffffff; padding: 24px; border: 1px solid
#e0e0e0; border-top: none; border-radius: 0 0 12px 12px;">
        <h2 style="margin: 0 0 16px; font-size: 18px; color: #333;">Новый
заказ-наряд #${orderId}</h2>
        <p style="color: #666; margin: 0 0 12px;">Автомобиль:
<strong>${vehicleInfo}</strong></p>
        <div style="background: #5c6bc015; border-left: 4px solid #5c6bc0;
padding: 12px 16px; border-radius: 0 8px 8px 0; margin: 16px 0;">
          <p style="margin: 0; color: #333;">Ваш автомобиль принят в работу.
Мы уведомим вас об изменении статуса.</p>
        </div>
        <hr style="border: none; border-top: 1px solid #eee; margin: 20px
0;" />
        <p style="color: #999; font-size: 13px; margin: 0;">Это
автоматическое уведомление. Не отвечайте на это письмо.</p>
      </div>
    </div>
  `;

  await transport.sendMail({
    from: fromAddress,
    to: toEmail,
    subject: `Новый заказ #${orderId} - АвтоСервис`,
    html,
  });
}

```

Листинг 4 – rateLimit

```

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 минут
  max: 10, // максимум 10 попыток за окно
  message: { error: 'Слишком много попыток. Повторите через 15 минут.' },
  standardHeaders: true,
  legacyHeaders: false,
});

const router = Router();

```

```

router.post('/register', authLimiter, register);
router.post('/login', authLimiter, login);
router.get('/me', authenticate, getMe);

```

Листинг 5 – Маршрутная таблица

```

function App() {
  return (
    <ThemeProvider>
      <CssBaseline />
      <BrowserRouter>
        <AuthProvider>
          <Routes>
            <Route path="/login" element={<PublicRoute><LoginPage
/></PublicRoute>} />
            <Route path="/register" element={<PublicRoute><RegisterPage
/></PublicRoute>} />

            <Route element={<ProtectedRoute><MainLayout
/></ProtectedRoute>}>
              <Route path="/dashboard" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.MANAGER]}>
                  <DashboardPage />
                </ProtectedRoute>
              } />
              <Route path="/clients" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.MANAGER,
Role.MASTER]}>
                  <ClientsPage />
                </ProtectedRoute>
              } />
              <Route path="/vehicles" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.MANAGER,
Role.MASTER]}>
                  <VehiclesPage />
                </ProtectedRoute>
              } />
              <Route path="/orders" element={<OrdersPage />} />
              <Route path="/orders/kanban" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.MANAGER,
Role.MASTER]}>
                  <KanbanPage />
                </ProtectedRoute>
              } />
              <Route path="/orders/new" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.MASTER]}>
                  <OrderCreatePage />
                </ProtectedRoute>
              } />
              <Route path="/orders/:id" element={<OrderDetailPage />} />
              <Route path="/services" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.MANAGER,
Role.MASTER]}>
                  <ServicesPage />
                </ProtectedRoute>
              } />
              <Route path="/parts" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.STOCKMAN,
Role.MANAGER]}>
                  <PartsPage />
                </ProtectedRoute>
              } />
              <Route path="/suppliers" element={
                <ProtectedRoute roles={[Role.ADMIN, Role.STOCKMAN]}>

```

```

        <SuppliersPage />
      </ProtectedRoute>
    } />
    <Route path="/supplies" element={
      <ProtectedRoute roles={[Role.ADMIN, Role.STOCKMAN]}>
        <SuppliesPage />
      </ProtectedRoute>
    } />
    <Route path="/employees" element={
      <ProtectedRoute roles={[Role.ADMIN]}>
        <EmployeesPage />
      </ProtectedRoute>
    } />
    <Route path="/reports" element={
      <ProtectedRoute roles={[Role.ADMIN, Role.MANAGER]}>
        <ReportsPage />
      </ProtectedRoute>
    } />
    <Route path="/notifications" element={
      <ProtectedRoute roles={[Role.CLIENT]}>
        <NotificationsPage />
      </ProtectedRoute>
    } />
    <Route path="/profile" element={<ProfilePage />} />
  </Route>

  <Route path="/" element={<ProtectedRoute><HomeRedirect
/></ProtectedRoute>} />
  <Route path="*" element={<Navigate to="/" />} />
</Routes>
</AuthProvider>
</BrowserRouter>
<PWAUpdatePrompt />
</ThemeContextProvider>
);
}

```

Листинг 6 – Axios

```

import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);

```

```
export default api;
```

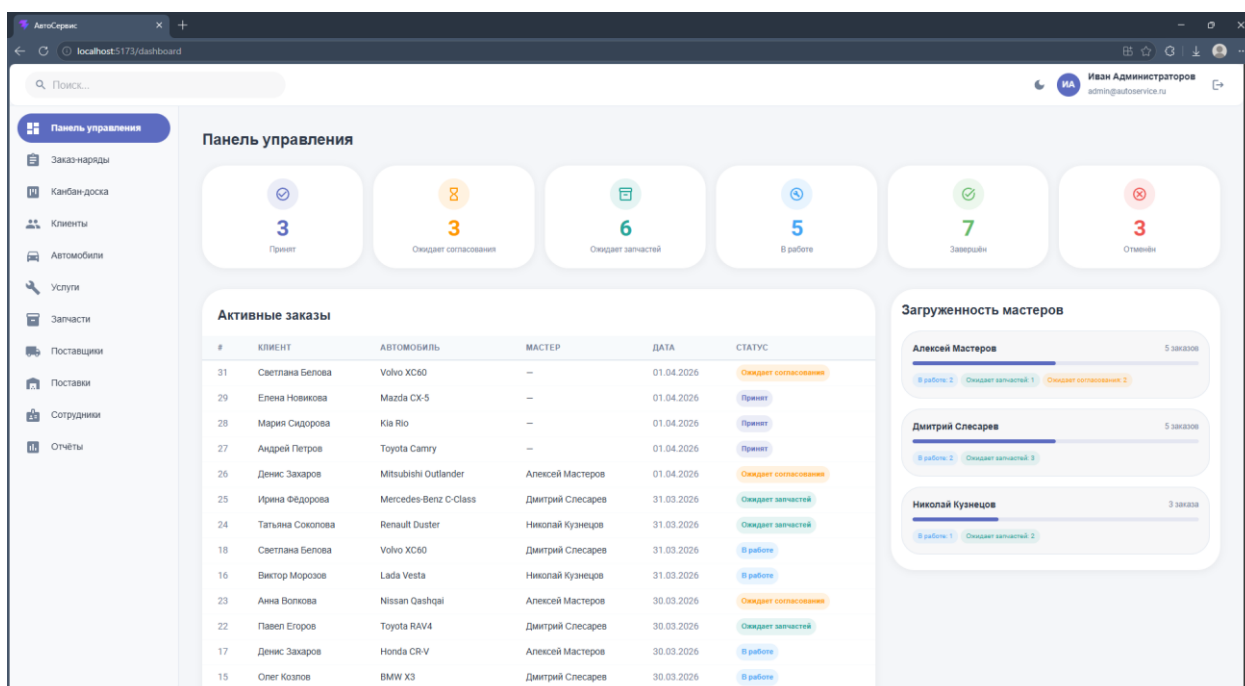


Рисунок 3 – Панель управления. Демонстрация клиентской части

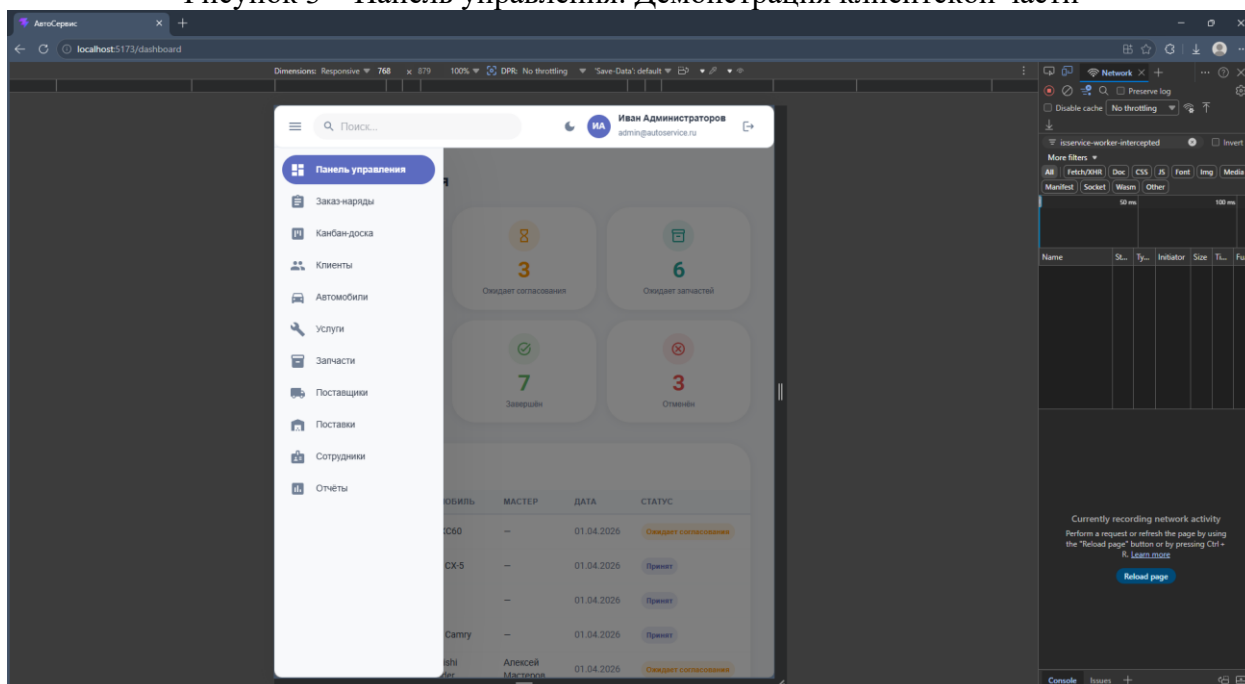


Рисунок 4 – Панель управления. Демонстрация клиентской части

Файл тестирования	Количество тест-кейсов	Проверяемые сценарии
auth.test.ts	8	Регистрация, авторизация, получение профиля, отклонение дубликатов, неверных паролей, отсутствующих и невалидных токенов
orders.test.ts	9	Создание заказа, получение списка и деталей, смена статуса с комментарием, запись в истории статусов, фильтрация по статусу, проверка доступа клиента к своему заказу, обработка несуществующих ID
rbac.test.ts	15	Ролевое разграничение доступа к Dashboard, клиентам, сотрудникам, заказам, отчётам и запчастям для ролей ADMIN, MASTER, CLIENT и неавторизованных запросов
parts.test.ts	6	CRUD запчастей: создание, получение списка и деталей, обновление, удаление неиспользуемой, обработка несуществующих ID
health.test.ts	4	Проверка эндпоинта мониторинга /api/health: статус БД, статистика, информация о сервере

Таблица 1 – Таблица со всеми тестами